

Introduction to High Performance Machine Learning

Lecture 2 02/02/26

Dr. Kaoutar El Maghraoui

ML Performance Optimization Methodology

Outline

- ML Performance Factors
- Performance Optimization Methodology:
 1. Measurement: Metrics, Time/Resources and Techniques
 2. Analysis: Amdahl's Law, Bandwidth/Latency, Roofline Model
 3. Optimization (in relationship to Roofline model)
- PyTorch Optimizer Algorithms
- PyTorch Multiprocessing
- PyTorch DataLoader and Disk performance

System vs. Algorithmic view

ML Performance Factors

Algorithm Performance

- Training vs. Inference Performance
- Distributed Training Algorithms. Ex: Asynchronous vs. Synchronous Distributed SGD
- Algorithms for Efficient Inference. Ex: Quantization and Pruning

Hyperparameters Performance

- ex: Learning rate, Network dimensions, Batch size, etc.

Implementation Performance

- Algorithms Implementation (vectorization, comm./comp. overlapping, parallelization, data access)

Framework Performance

- ML Frameworks: PyTorch, TensorFlow, Caffe, MXNET

Library Performance

- Math libraries (cuDNN), Communication Libraries (MPI, NCCL, GLU)

Hardware Performance

- CPU, DRAM, GPU, HBM, Tensor Units, Disk/Filesystem, Network

System view:
Bottom-Up
(this course)



Algorithmic view:
Top-down



System vs. Algorithmic View

Bottom-Up



Systems View

Focus on **Hardware Utilization** and efficiency.

Optimizing **Memory Bandwidth** and data movement.

Instruction-level efficiency and **Kernel Performance**.

Top-Down



Algorithmic View

Focus on **Loss Curves** and convergence rates.

Model **Architecture** and hyperparameter tuning.

Mathematical correctness and **Generalization**.

The Bridge

High-Performance ML (HPML) engineers care about the **utilization beneath the loss curve**.

Algorithmic Performance Examples

Example 1 — Dense Attention vs FlashAttention

- Dense attention:
 - $O(N^2)$ memory traffic
- FlashAttention:
 - $O(N^2)$ compute
 - $O(N)$ memory

➡ Same asymptotic FLOPs, radically different real runtime due to memory behavior.

Key Takeaway:

Algorithmic design decisions strongly influence memory traffic, computation patterns, and execution structure

Example 2 — Full Decode vs Speculative Decoding

- Baseline:
 - One forward pass per generated token
- Speculative:
 - Small model proposes K tokens
 - Large model verifies in one pass

➡ **Speedup:** $2\times - 4\times$

Example 3 — Dense Model vs Mixture-of-Experts

- Dense 70B:
 - All parameters active per token
- MoE:
 - Only 2–4 experts active

➡ Similar quality, **much lower compute per token**

Implementation Performance Examples

Example 1 — Naive Attention vs Fused FlashAttention

- Naive PyTorch attention implemented with separate:
 - QK^T matmul, softmax, dropout
 - V matmul
- Causes:
 - Multiple kernel launches
 - Large intermediate tensors
 - Excessive HBM traffic
- FlashAttention (fused kernel):
 - Single kernel
 - Tiling + recomputation
 - No large intermediate storage

➡ **Speedup:** 2× – 6× faster, lower memory usage

Example 2 — Python Loop vs Vectorized Tensor Ops

- Looping over tokens in Python:

```
for i in range(T):  
    out[i] = W @ x[i]
```

- Vectorized Implementation:

```
out = X @ W.T
```

➡ **Speedup:** 2× – 6× faster, lower memory usage

Example 3: Implementation Performance in DL

- **Implementation Performance:**
 - Using inefficient memory operations during the forward or backward pass in a neural network (e.g., using unoptimized GPU memory allocations in PyTorch) can cause significant slowdowns, similar to the impact of too many malloc() operations in C.
 - **Example:**
 - Recomputing activations in each training step instead of storing them efficiently may lead to performance bottlenecks.
 - Unnecessary tensor allocations

PyTorch Example

Inefficient Memory Operations

```
1 import torch
2 import torch.nn as nn
3
4 # Define a simple neural network
5 class InefficientNet(nn.Module):
6     def __init__(self, input_size, hidden_size, output_size):
7         super(InefficientNet, self).__init__()
8         self.fc1 = nn.Linear(input_size, hidden_size)
9         self.fc2 = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         # Inefficient memory usage: allocating new tensors in each forward pass
13         x = torch.relu(self.fc1(x))
14
15         # Instead of reusing 'x', we create a new tensor which consumes more memory.
16         # The new tensor is allocated every time this forward method is called.
17         new_tensor = torch.relu(self.fc2(x))
18
19         # Returning the newly created tensor
20         return new_tensor
21
22 # Dummy data
23 input_data = torch.randn(64, 100)
24
25 # Model instance
26 model = InefficientNet(100, 50, 10)
27
28 # Forward pass (new tensors are allocated every time)
29 output = model(input_data)
30
```

Efficient Version

```
1 import torch
2 import torch.nn as nn
3
4 # Define a more efficient neural network
5 class EfficientNet(nn.Module):
6     def __init__(self, input_size, hidden_size, output_size):
7         super(EfficientNet, self).__init__()
8         self.fc1 = nn.Linear(input_size, hidden_size)
9         self.fc2 = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         # Efficient memory usage: reusing the same tensor without creating new ones
13         x = torch.relu(self.fc1(x))
14
15         # Reusing the existing tensor instead of allocating a new one
16         x = torch.relu(self.fc2(x))
17
18         return x
19
20 # Dummy data
21 input_data = torch.randn(64, 100)
22
23 # Model instance
24 model = EfficientNet(100, 50, 10)
25
26 # Forward pass (memory reuse)
27 output = model(input_data)
28
```

PyTorch Example

Inefficient Memory Operations: Creating new tensors repeatedly increases memory traffic and pressure on the GPU memory allocator

```
1 import torch
2 import torch.nn as nn
3
4 # Define a simple neural network
5 class InefficientNet(nn.Module):
6     def __init__(self, input_size, hidden_size, output_size):
7         super(InefficientNet, self).__init__()
8         self.fc1 = nn.Linear(input_size, hidden_size)
9         self.fc2 = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         # Inefficient memory usage: allocating new tensors in each forward pass
13         x = torch.relu(self.fc1(x))
14
15         # Instead of reusing 'x', we create a new tensor which consumes more memory.
16         # The new tensor is allocated every time this forward method is called.
17         new_tensor = torch.relu(self.fc2(x))
18
19         # Returning the newly created tensor
20         return new_tensor
21
22 # Dummy data
23 input_data = torch.randn(64, 100)
24
25 # Model instance
26 model = InefficientNet(100, 50, 10)
27
28 # Forward pass (new tensors are allocated every time)
29 output = model(input_data)
30
```

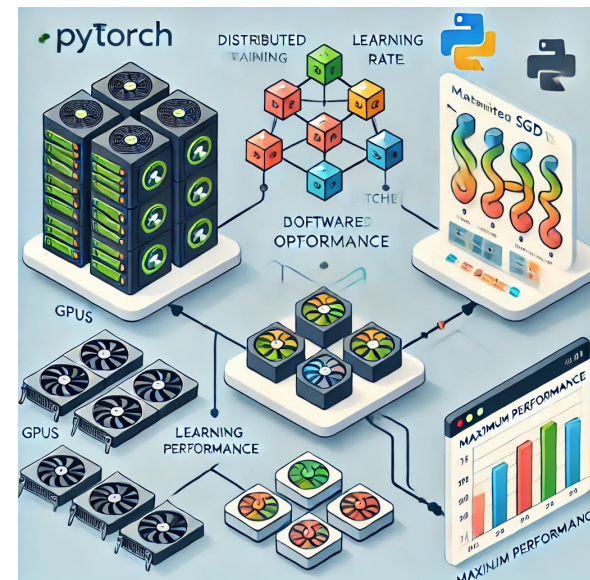
Efficient Version: PyTorch's memory allocator reuses previously allocated buffers

```
1 import torch
2 import torch.nn as nn
3
4 # Define a more efficient neural network
5 class EfficientNet(nn.Module):
6     def __init__(self, input_size, hidden_size, output_size):
7         super(EfficientNet, self).__init__()
8         self.fc1 = nn.Linear(input_size, hidden_size)
9         self.fc2 = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         # Efficient memory usage: reusing the same tensor without creating new ones
13         x = torch.relu(self.fc1(x))
14
15         # Reusing the existing tensor instead of allocating a new one
16         x = torch.relu(self.fc2(x))
17
18         return x
19
20 # Dummy data
21 input_data = torch.randn(64, 100)
22
23 # Model instance
24 model = EfficientNet(100, 50, 10)
25
26 # Forward pass (memory reuse)
27 output = model(input_data)
28
```

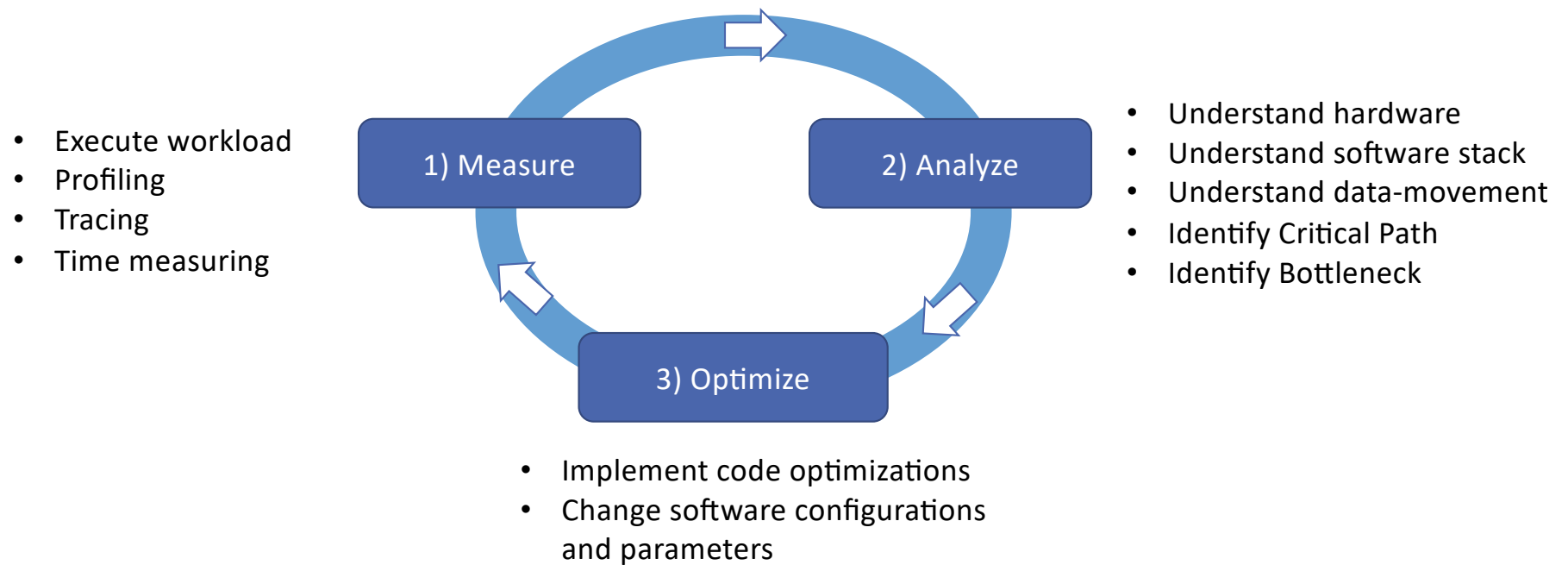
Software Performance Optimization

ML Performance Optimization Definition

- Software Performance Optimization for ML
 - Given:
 - A **system** (ex: A Cluster Compute node + PyTorch)
 - An **algorithm** (ex: Distributed SGD training) + **hyperparameters**
 - A **dataset** (ex: CIFAR100)
 - Obtain the **maximum** performance



Performance Optimization Methodology



Performance optimization methodology (1): Measurement

What is performance?

- Basic metrics:

- Execution time: t (for a single operation is called **latency**)

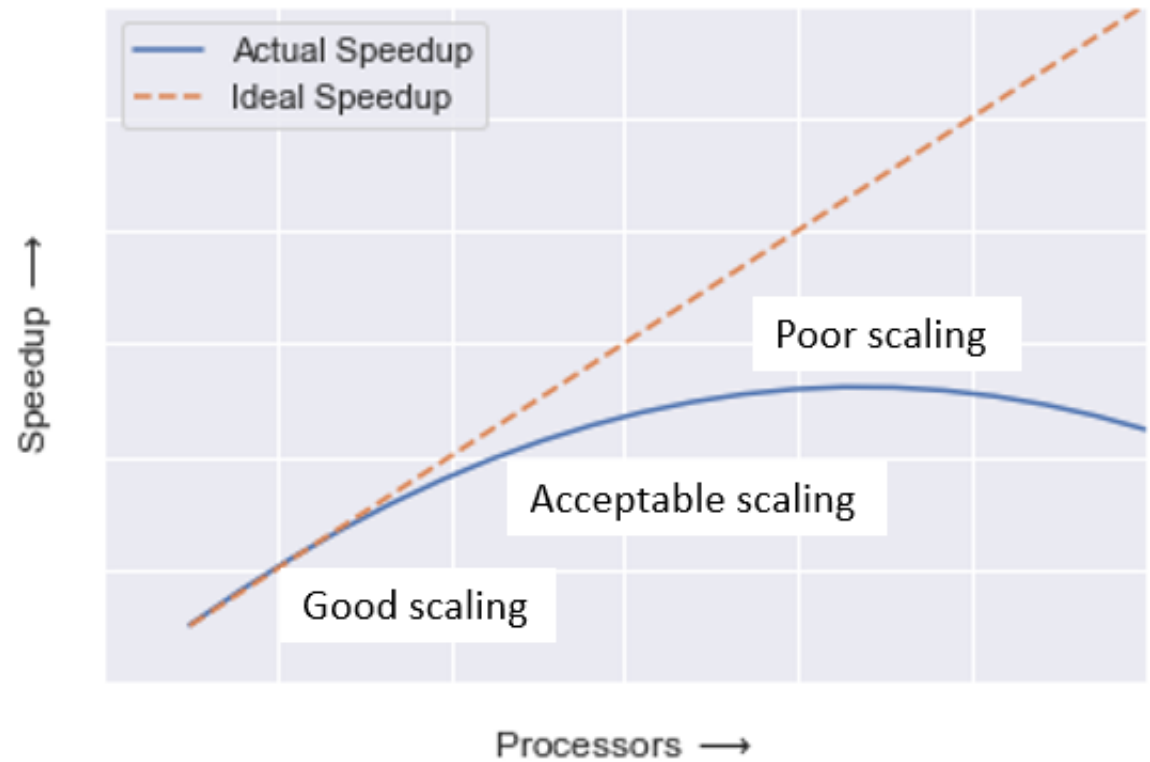
- Derived metrics:

- Throughput: $\frac{\# \text{ operations}}{t}$ or $\frac{\# \text{ programs}}{t}$

- FLOPS: $\frac{\# \text{ floating_point_operations}}{t}$ (<https://en.wikipedia.org/wiki/FLOPS>)

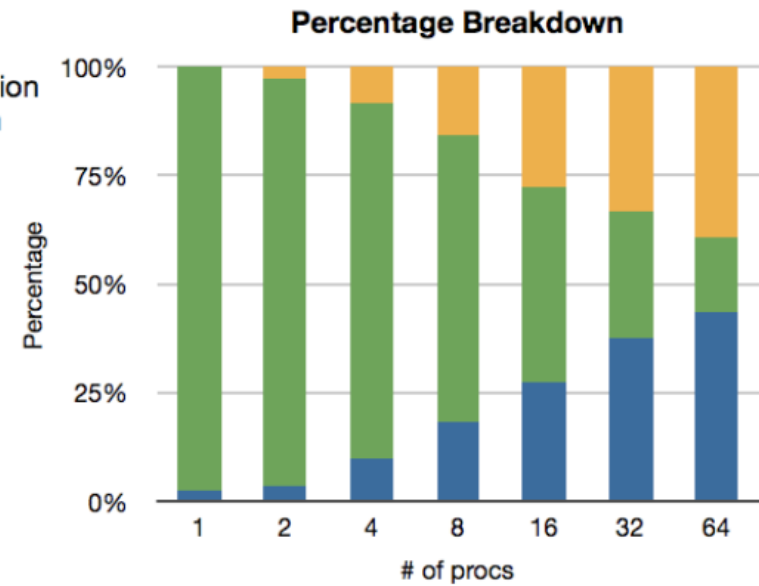
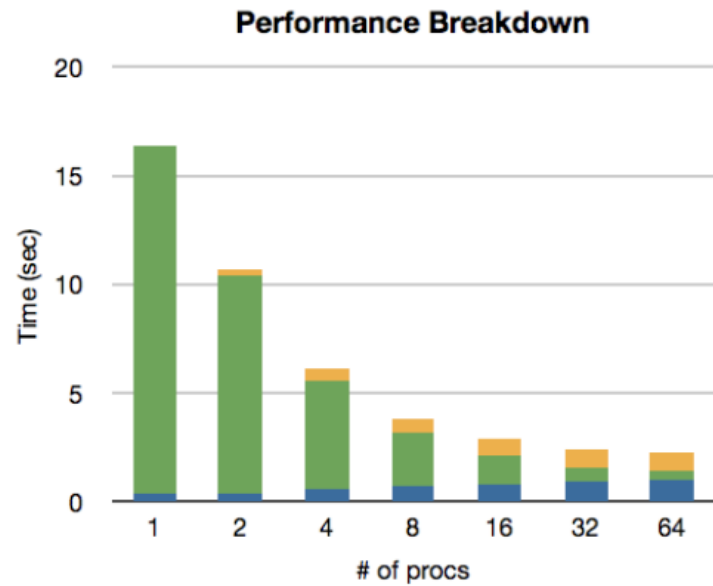
Speedup

- Speedup of B w.r.t. A: $\frac{t_A}{t_B}$
- Parallel Speedup: $\frac{t_{serial}}{t_{parallel}}$
- Slowdown is inverse of Speedup



From: <https://waterprogramming.wordpress.com/tag/weak-scaling/>

“Not Good” speedup



Scalability

- **Scaling Efficiency**

- $E = \frac{t_{serial}}{t_{parallel} * p} \leq 1$ p is the number of processes/threads/...

- **Strong Scaling:** Constant problem size while increasing p

- **Definition:** Measures how well a system performs a fixed problem size as the number of resources (e.g., processors, GPUs) increases.
 - **Goal:** Reduce the time to solve the same problem by adding more computational resources.
 - How the solution time varies with the number of processors for a fixed total problem size.
 - Increasing synchronization cost, but fixed amount of work

- **Weak Scaling:** Increasing problem size proportional to p

- **Definition:** Measures how well a system performs as both the workload size and the number of resources increase proportionally.
 - **Goal:** Keep the runtime constant as the workload grows in tandem with the resources.
 - Weak scaling is defined as how the solution time varies with the number of processors for a fixed problem size per processor.
 - Work per process is constant
 - Increasing synchronization cost, increasing work

- While **speedup** tells us how much faster something becomes, **scalability** tells us **how well** that speedup holds as we continue adding resources.
- Scaling efficiency measures how **close we are to ideal linear scaling**.

Weak vs. Strong Scaling

Assume Serial program that solves a problem size P in time T

E.g., Protein folding simulation in an hour

Weak Scaling

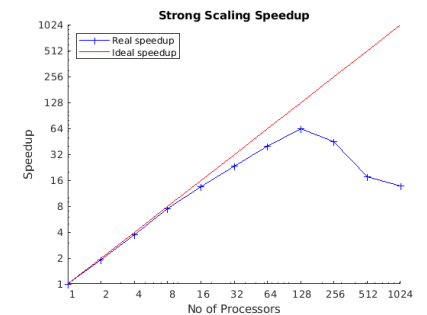
Weak scaling: run a larger problem or more problems within T

- E.g., fold a bigger protein or more small ones in an hour

Strong Scaling

Strong scaling: run a problem faster than T

- E.g. fold the same protein in a min



What Scaling?

- When my problem continues to increase in size, I can still solve the problem within the same amount of time by simply dedicating proportionally more resources to it.
- When my problem stays the same size, I can solve the problem 10 times faster by dedicating 10 times more resources.

Deep Learning Data Parallelism: Strong Scaling

- **Scenario:** We fix a batch size and vary the number of CPUs/GPUs to train a neural network.
- **Example:** Given a batch size of 1024 training samples, an eight-GPU system will give us the following benchmarking configurations
- **The efficiency of strong scaling** is calculated in the following manner:
 - $\text{efficiency} = t_1 / (N * t_N) * 100\%$
 - t_1 : time to solve a problem with one compute device
 - N : number of compute devices
 - t_N : time to solve the same problem with N devices
- **Challenges:** Communication and synchronization overhead

#GPUs	GPUs IDs	per-GPU batch size	effective batch size
1	0	1024	1024
2	0,1	512	1024
4	0,1,2,3	256	1024
8	0,1,2,3,4,5,6,7	128	1024

Deep Learning Data Parallelism: Weak Scaling

- **Scenario:** we keep per-device batch size fixed
- **Example:** We get the following benchmarking configurations assuming that the per GPU batch size is 128 and the same hardware setup:
- **The efficiency of strong scaling** is calculated in the following manner:
 - $\text{efficiency} = t_1 / (t_N) * 100\%$
 - t_1 : time to solve a problem with one compute device
 - N: number of compute devices
 - t_N : time to solve the N times larger problem with N compute devices
- **Challenges:** this could lead to huge batch sizes when convergence suffers leading to slowdown

#GPUs	GPUs IDs	per-GPU batch size	effective batch size
1	0	128	128
2	0,1	128	256
4	0,1,2,3	128	512
8	0,1,2,3,4,5,6,7	128	1024

Computing Averages – Mean Values

- Average Execution Time

- Arithmetic mean: $\frac{1}{n} \sum_{i=1}^n t_i$

- Average Performance or Throughput

- If t is held constant => Arithmetic mean
- If $\#operations$ is held constant => Harmonic mean:

$$\frac{n}{\sum_{i=1}^n \frac{t_i}{\#operations}}$$

- Average Speedup, Slowdown or any Ratio

- Geometric mean: $\sqrt[n]{\prod_{i=1}^n speedup_i}$

Which mean to use?

Arithmetic Mean

$$\overline{x}_A = \frac{1}{n} \sum_{i=1}^n x_i$$

- Produces a mean value that is *directly proportional to the total time*
- When the sum of raw values has physical meaning
- Use for summarizing **times** (not rates)
- Use when measuring values that contribute equally to the final result, such as in time measurements or performance in identical workloads.

Harmonic Mean

$$\overline{x}_H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

- Produces a mean value that is proportional to the sum of the inverse of times
- Use for averaging rates like throughput or speed (not times), where the individual measurements are inversely related to the total performance
- Examples: F1 score in Machine Learning (harmonic mean of precision and recall)
- Example: Network Throughput: Often used when measuring avg throughput across multiple links

Geometric Mean

$$\begin{aligned} \overline{x}_G &= \sqrt[n]{x_1 x_2 \cdots x_i \cdots x_n} \\ &= \left(\prod_{i=1}^n x_i \right)^{1/n} \end{aligned}$$

- Use when numbers in the series are not independent or if numbers tend to make large fluctuations.
- Used when comparing ratios or proportions such as growth rates or performance scaling across systems.
- They are used when data points have different units or scales.
- Example: Benchmark performance across a variety of workloads.
- Example: Financial systems to calculate avg investment returns

Summary

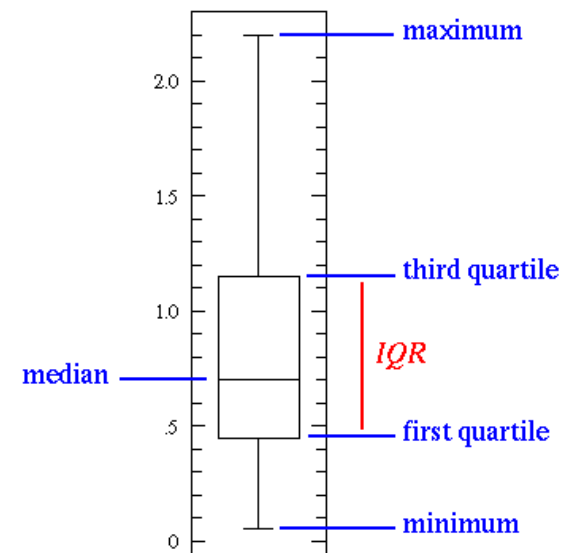
Mean Type	Formula	Best for	Example Usage
Arithmetic Mean	$\frac{1}{n} \sum_{i=1}^n x_i$	Additive data	CPU performance, latency measurements
Harmonic Mean	$\frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$	Averaging rates, reciprocal values	Network throughput, F1 score in machine learning
Geometric Mean	$(\prod_{i=1}^n x_i)^{\frac{1}{n}}$	Multiplicative data, ratios and proportions	Performance benchmarking, financial returns, scalability

Benchmarking Workloads

- Benchmarks in ascending order of complexity:
 1. Micro-kernels: test a specific processor feature
Examples: Floating point, L1 Cache, L2 Cache,
 2. Micro-benchmark: small program from a programming assignment
Examples: Merge sort in isolation
 3. Kernels: a specific algorithm in a real program
Examples: Quicksort, Binary Search, DGEMM (Dense Generalized Matrix Multiplications)
 4. Synthetic Benchmarks: try to reproduce the workload of a class of applications
Examples: [Dhrystone](#) (concentrates on string handling) and Linpack (solves a dense system of linear equations)
 5. Real Applications: a real application used for a specific purpose
Examples: Word, MySQL, NAMD (Molecular Dynamics), [MLPerf Inference benchmark](#)
 6. Real Workflows: a set of applications working together
Example: [CANDLE](#) workflow
(a workflow framework for machine learning applied to cancer research)

Measuring and Reporting Performance

- Reproducibility
 - Always include absolute execution time
 - Report relevant hardware and software info:
 - CPU, Memory, Network, Disk, etc.
 - Experiment configuration
 - Code, Pseudo code
 - Compiler ver., Compilation Flags, Libraries ver., OS ver.
- Accuracy
 - Repetitions: 5, 10, 100, ... (depends on variability)
 - If high-variability results:
 - Try to understand why and reduce it
 - Include stddev, variance, max-min, inter-quartile range
 - Use box-plot for chart representation as shown in figure

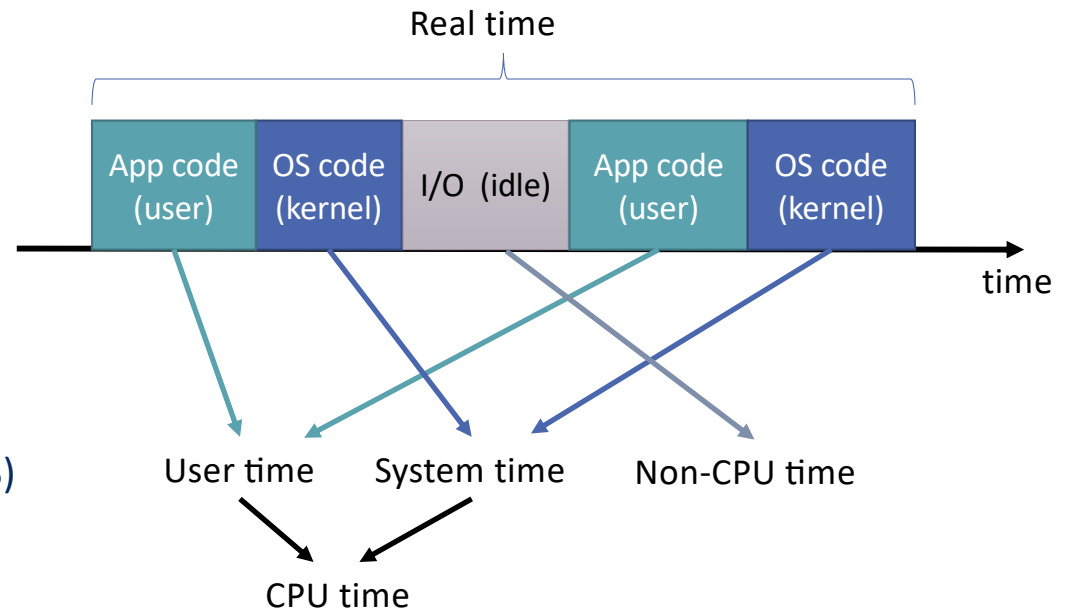


Basic and advanced measurement techniques

- Basic:
 - Time measurement
 - Application Throughput
 - Breakdown phases or iterations
- Advanced:
 - Profiling
 - Tracing

Time definitions

- **Real (or Wall Clock or Elapsed) Time** : actual elapsed time from a point in the past
- **CPU (or Process) Time**: time spent executing CPU instructions
 - **User Time** : time spent in user space
 - **System Time** : time spent in kernel space (OS)
- **Non-CPU Time**: time spent waiting (idle CPU) for: I/O, Virtualization, etc.



https://en.wikipedia.org/wiki/CPU_time

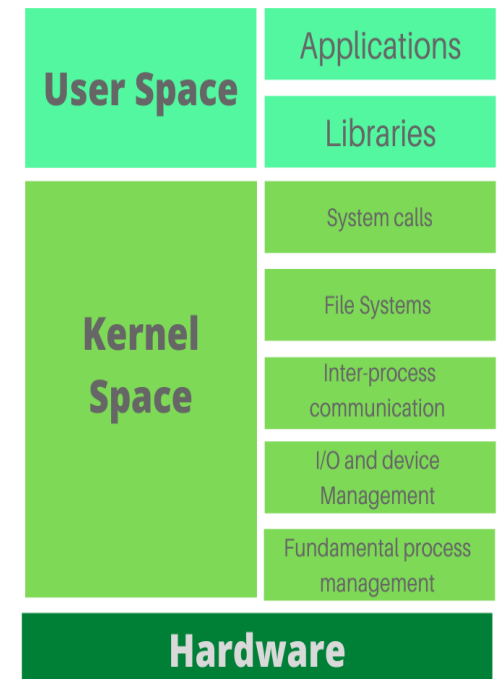
Time Measurement - Linux

- *time* command - Real, User and System times

```
$ time ./executable
```

```
real    0m1.057s
user    0m1.015s
sys     0m0.000s
```

- millisecond granularity, accuracy may vary between systems!
- $\text{real} \geq \text{user} + \text{sys}$



User Space vs Kernel Space

User Space

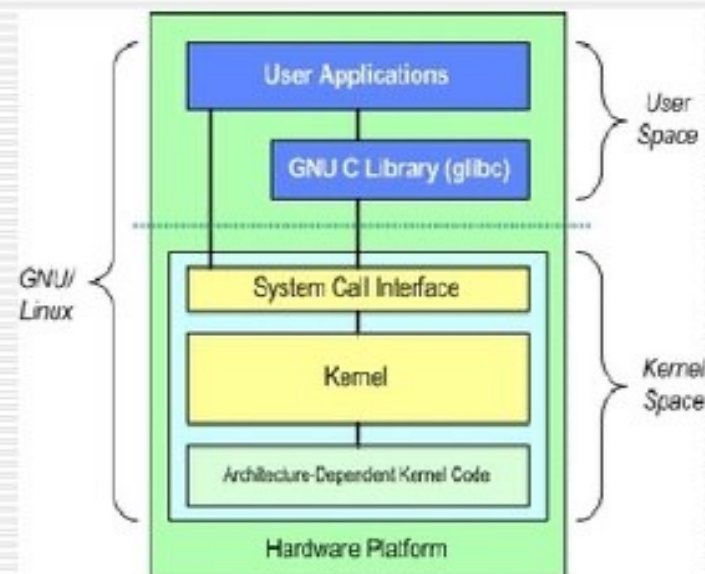
□ User space is the memory area where all user mode application work and this memory can be swapped out when necessary

□ User space process normally runs in its own virtual memory space and unless explicitly requested, cannot access the memory of other processes.

Kernel Space

□ Kernel Space is strictly reserved for running the kernel (OS background process), kernel extensions and most device drivers

□ Linux kernel space gives full access to the hardware, although some exceptions run in user space. (The graphic system most people use with Linux does not run in kernel in contrast to that found in Microsoft Windows)



Time measurement in C

- `clock_gettime(CLOCK_MONOTONIC,..)` - Real time
 - Nanosecond granularity - measuring in usec:

```
#include <time.h>
struct timespec start, end;

clock_gettime(CLOCK_MONOTONIC, &start);
<CODE TO MEASURE>
clock_gettime(CLOCK_MONOTONIC, &end);

double time_usec = (((double)end.tv_sec * 1000000 + (double)end.tv_nsec / 1000)
    - ((double)start.tv_sec * 1000000 + (double)start.tv_nsec / 1000));
printf("a=%d time: %.03lf\n", a, time_usec);
```

- <http://btorpey.github.io/blog/2014/02/18/clock-sources-in-linux/>

Execution Time measurement in Python

- Real Time:

- granularity fractions of seconds – printing in seconds (Python 3.3)

```
import time

start=time.monotonic()
<CODE TO MEASURE>
end=time.monotonic()

print("time: " + str(end-start))
```

- From Python 3.7: *time.monotonic_ns()* (granularity in nanoseconds)
- <https://docs.python.org/3.7/library/time.html>

Example: Measuring Time Per Epoch in PyTorch

To measure the **execution time** for each epoch during PyTorch training, you can use the time module:

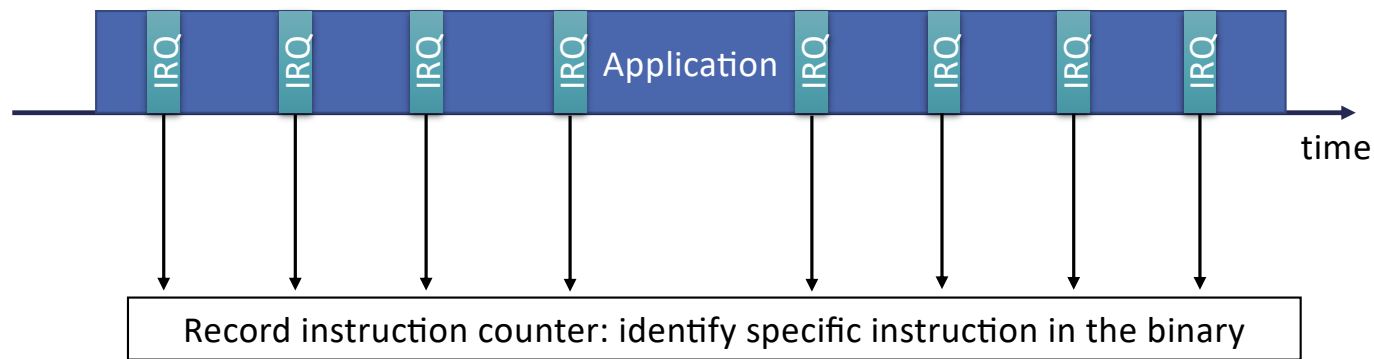
- **Import the time module** to track the start and end times.
- **Start a timer** at the beginning of each epoch using `time.time()`.
- **Compute the epoch duration** by subtracting the start time from the current time at the end of the epoch.

```
1
2  import time # Import time module
3  epochs = 5
4  for epoch in range(epochs):
5      start_time = time.time() # Start time for epoch
6
7      # Training loop here (forward, backward, and optimization)
8
9      epoch_time = time.time() - start_time # Time elapsed
10     print(f"Epoch [{epoch + 1}/{epochs}], Time Elapsed: {epoch_time:.2f} seconds")
11
```

Profiling

- Sampling:
 - Sample applications during execution to infer a statistical distribution:
Example: approximate time spent in each instruction of the code
- Counting:
 - Count exact events
 - Software counters (implemented in kernel): count specific events
 - Example: count number of memory allocations (malloc())
 - **Hardware performance counters (aka Performance Counters)**
 - Counters maintained in registers
 - Examples: count number of L2 misses, Floating-point ops, Integer ops, number of branch mispredictions

Profiling - Sampling



- IRQ (interrupt request): interruption of the application to execute a different routine
- Profiling uses IRQs to register instruction counter and other metrics at regular intervals
- Relatively low-overhead, depending on IRQ frequency

Tracing

```
int main(int argc, char **argv) {  
    RECORD_TRACE_EVENT("after_main");  
    struct timespec start, end;  
    int i,a=1;  
    clock_gettime(CLOCK_MONOTONIC,&start);  
    RECORD_TRACE_EVENT("before_loop");  
    for ( i=0; i < 1000000000; i++) {  
        RECORD_TRACE_EVENT("in_loop");  
        a += a*i;  
    }  
    RECORD_TRACE_EVENT("after_loop");  
    clock_gettime(CLOCK_MONOTONIC,&end);  
    RECORD_TRACE_EVENT("before_return");  
    return 0;  
}
```



TRACE Example

usec	event
[000012]	after_main
[000013]	before_loop
[000021]	in_loop
[000024]	in_loop
...	...
[012122]	in_loop
[012132]	after_loop
[012223]	before_return

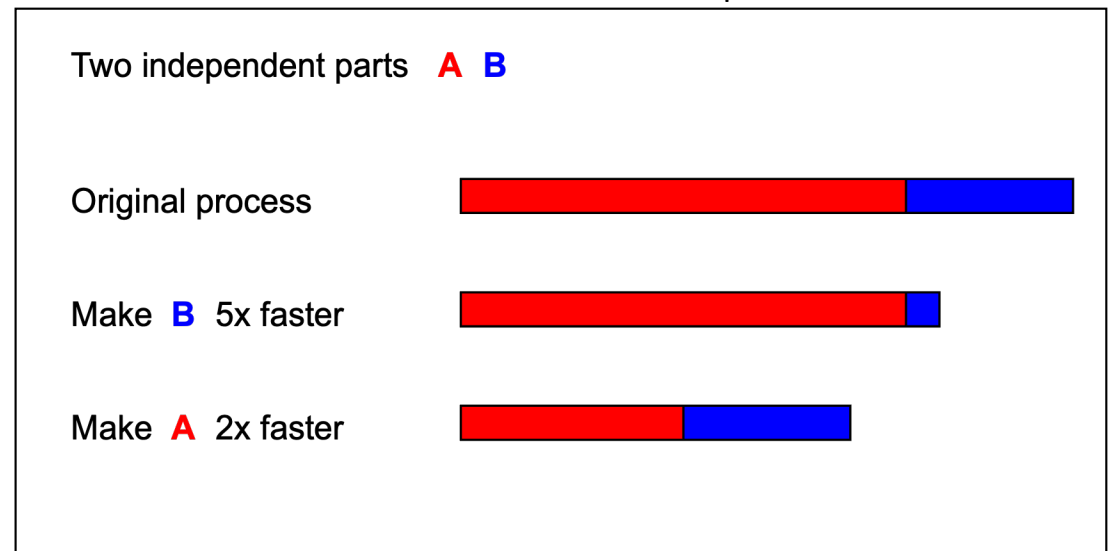
- Explicit code instrumentation with tracing primitives
- Higher overhead than profiling
- Linux perf tracing can be applied to any code: Applications, Runtime, Kernel, Etc.
- Tracing utilities: strace (trace system calls made by an application), ftrace (trace execution flow of kernel functions)

Performance optimization methodology (2): Analysis

Amdahl's Law

- $S(p, s) = \frac{1}{(1-p) + p/s}$
 - S : speedup of the entire application (or runtime, OS, etc.)
 - p : portion of the execution time that is spent in the code section before improvement (if time for p is high the section is called **critical section**)
 - s : speedup of the improved code section
- Overall speedup is limited by how much time the improved code takes compared to the rest

Amdahl's law effect example



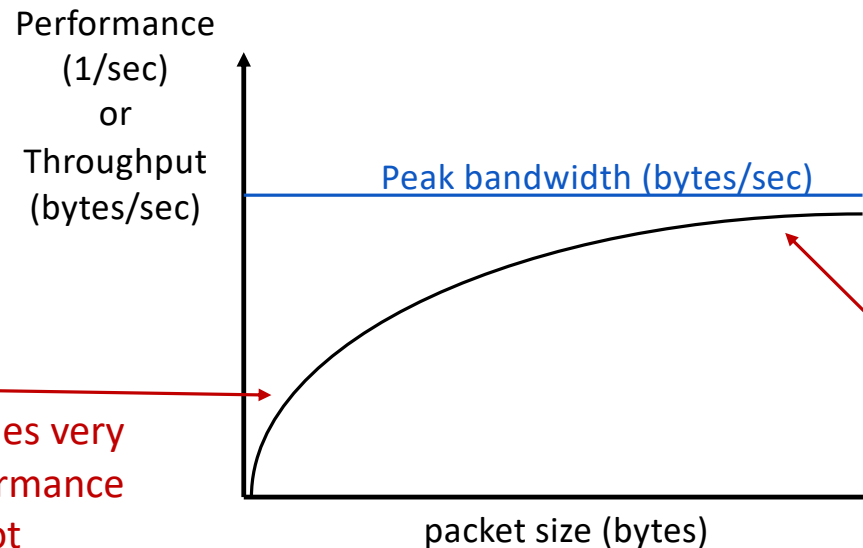
From: https://en.wikipedia.org/wiki/Amdahl%27s_law

Performance Analysis Step

1. Identify **Critical Path**: the section of the program that accounts for most of the time (high value of Amdahl's p)
 - Critical Path characteristics: very slow to execute (high latency) and/or executed many times
 - Use output of performance measurement step (profiling, tracing, etc.)
 - Verify hypothesis of critical path: comment code and run again
2. Identify the **Bottleneck**: the **system resource** that affects the execution time of the critical path
 - Need to understand software/hardware architecture
 - Bottleneck type: **Data Movement** vs. **Computation**

Data Movement and Packet Size

- True for any **Data Movement**: Network, PCIe, DRAM, etc.



- **SMALL Packets:**

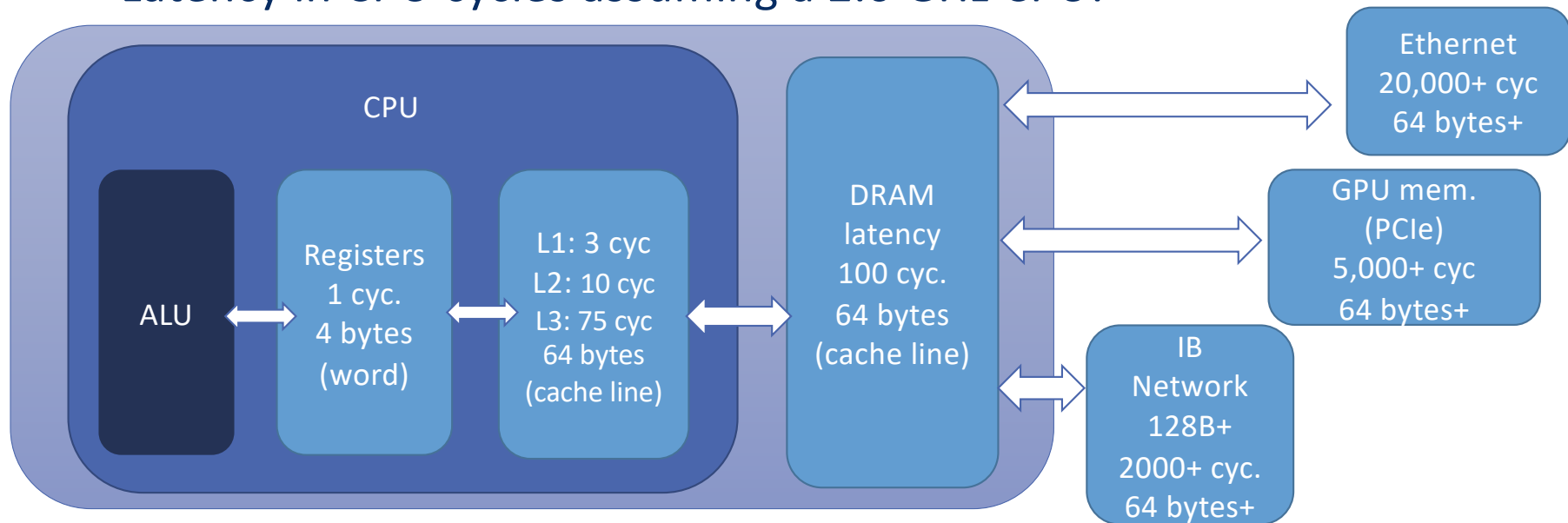
- **Low Latency** becomes very important for performance
- Full Bandwidth is not reached

- **LARGE Packets:**

- **High Bandwidth** becomes very important for performance
- Latency is not relevant

Data movement Locality Principle - Latency

- Latency in CPU cycles assuming a 2.0 GHz CPU:



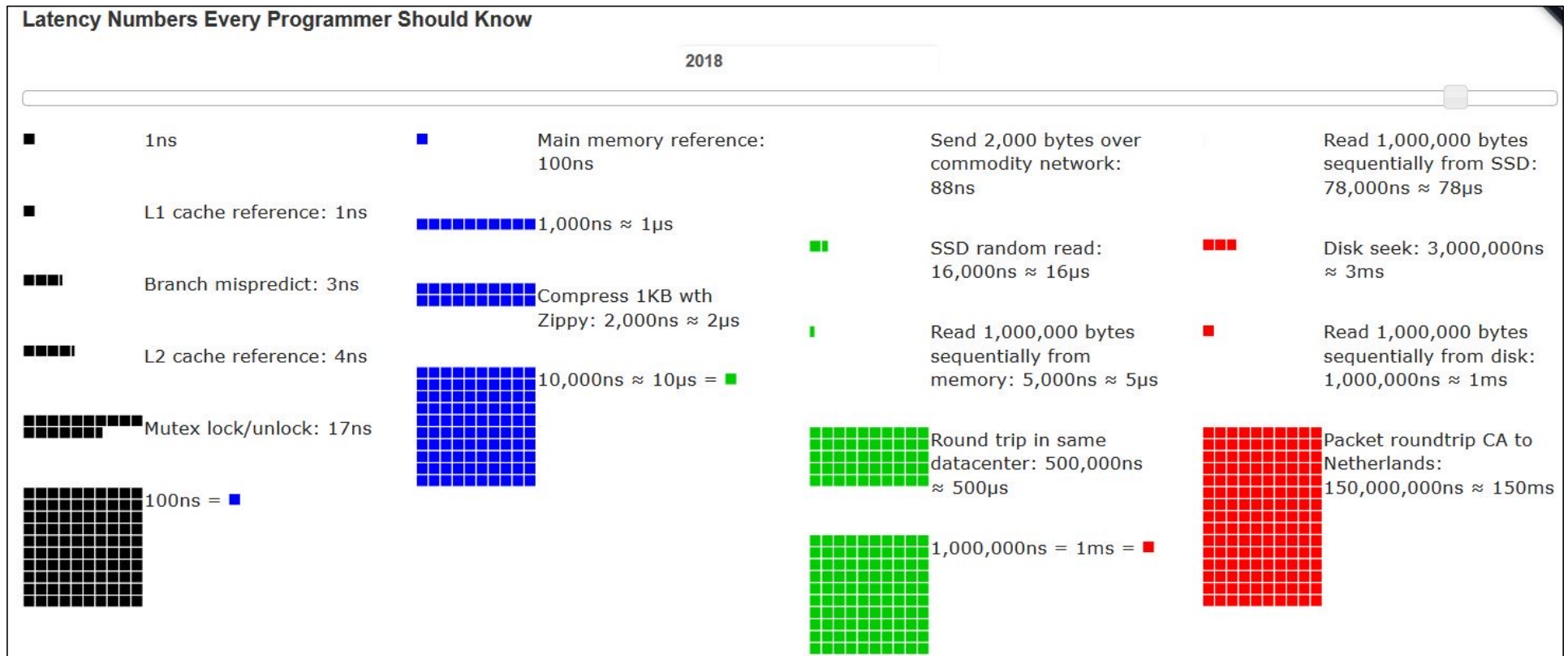
- small granularity
- low latency
- high bandwidth



- large granularity
- high latency
- low bandwidth

Latencies every programmer should know: [link](#)

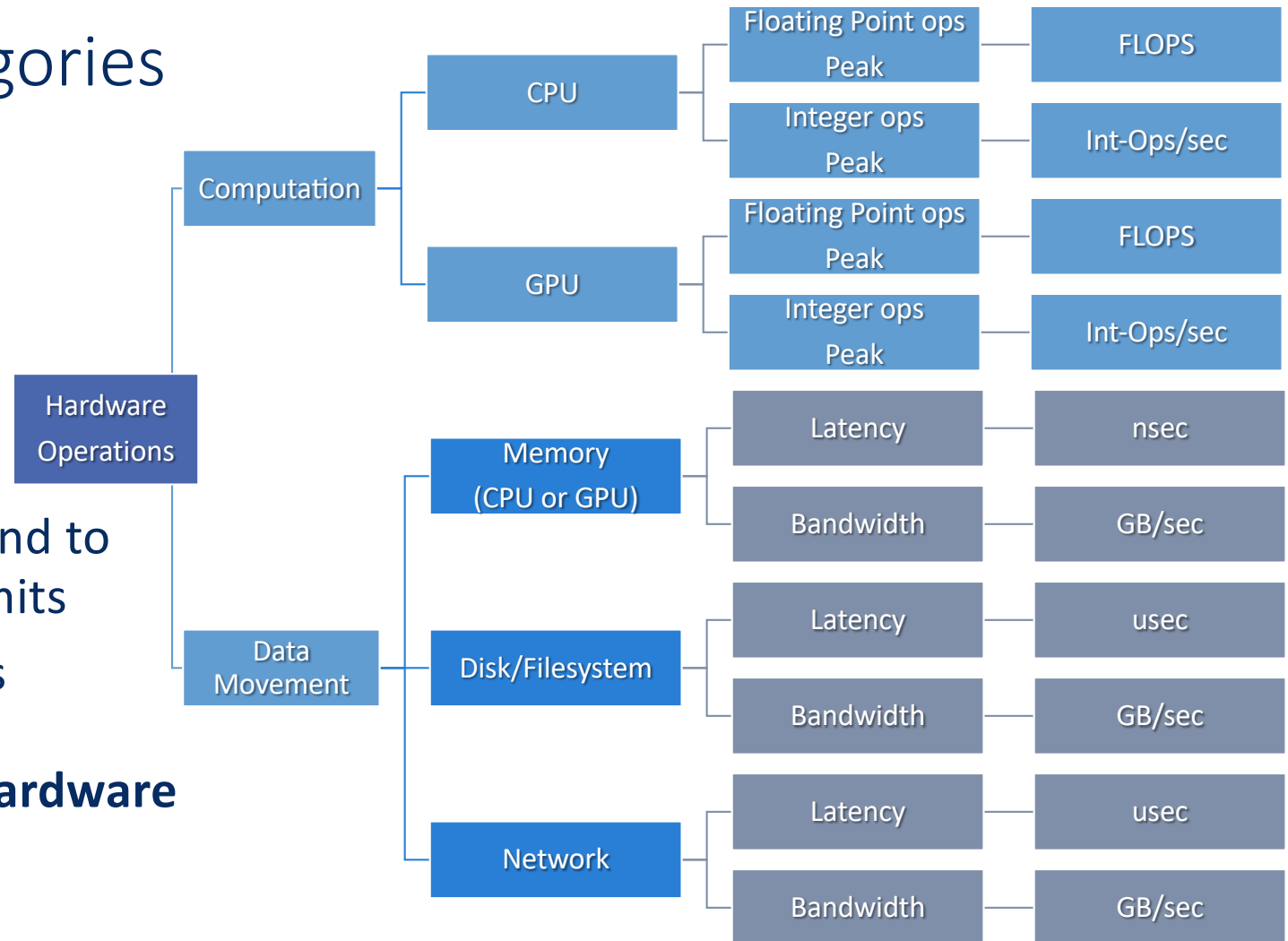
Latency values over the years – very cool tool!



- https://people.eecs.berkeley.edu/~rcs/research/interactive_latency.html

Bottleneck categories

- Bottlenecks correspond to specific Hardware Limits
- Performance Analysis Problem: **How far is performance from Hardware Limits?**



Performance Models Objectives

- Identify performance bottlenecks
- Determines Hardware Limits to Optimization
 - Determines how far we are from hardware limits
 - Motivate algorithmic changes
- Project performance on future hardware or applications

Peak FLOPS

- Peak FLOPS depend on:
 - Compute unit architecture: CPU, GPU, TPU, FPGA etc.
 - #cores and #threads
 - Clock Frequency
 - Precision: DP (64 bit), SP (32 bit), HP (16 bit)
 - SIMD instructions in the cores: Intel AVX, IBM AltiVec
- CPU formula for Peak FLOPS: $\#tot_cores \cdot \frac{cycles}{seconds} \frac{FLOPs}{cycles}$
- see <https://en.wikipedia.org/wiki/FLOPS>

Performance Model – Constants (HW specs)

- Examples of HW Specs for the performance model
 - **CPU peak DP/SP FLOPS:** GFLOPS/s
 - **DRAM peak Bandwidth:** GB/s
 - **GPU peak DP FLOPS:** TFLOPS/s
 - **HBM peak Bandwidth:** TB/s
- How to obtain:
 - Vendor hardware specifications
 - Alternative: run micro-benchmarks for compute and memory (lower bounds than specs)

Performance Model - Variables

- Actual Experimental Measurements :
 - Computation (CPU/GPU) Performance: FLOPS
 - Memory throughput: GB/s or TB/s
- How to measure:
 - FLOPS: **hw performance counters** for FLOP divided by time
 - GB/s: **hw performance counters** for memory-ops divided by time

Roofline Performance Model (1)

- Throughput-based model
- Developed at DOE Lawrence Berkeley Labs

- Metrics:

- Peak FLOPS

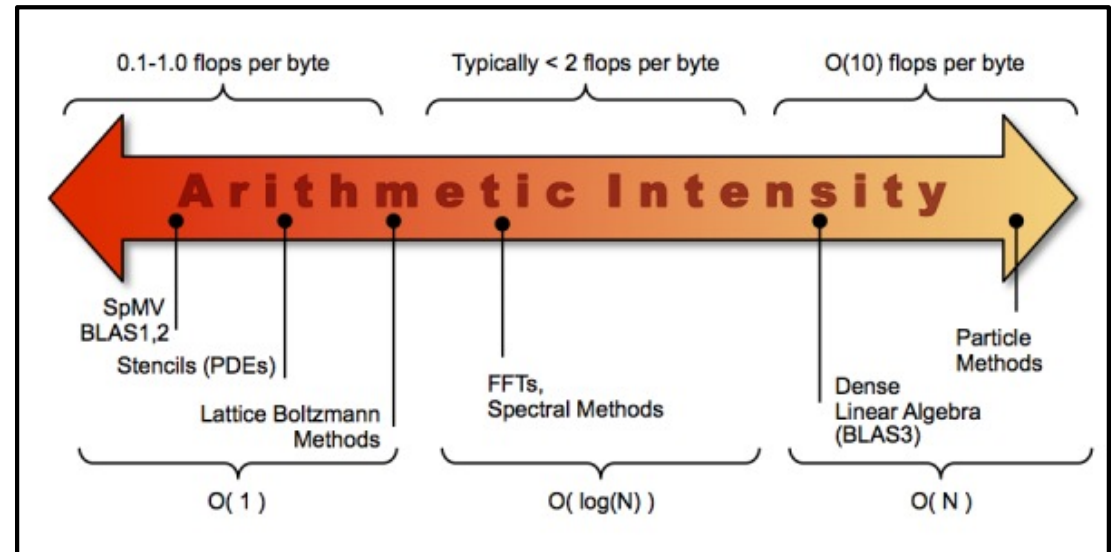
- Memory Bandwidth: $\frac{\text{data}}{\text{time}} \left[\frac{\text{bytes}}{\text{sec}} \right]$

- Arithmetic Intensity (program property):

$$\frac{\# \text{arithmetic ops}}{\text{DRAM data}} \left[\frac{\text{FLOP}}{\text{bytes}} \right]$$

(bytes as seen from DRAM)

note: FLOP $\neq$ FLOPS



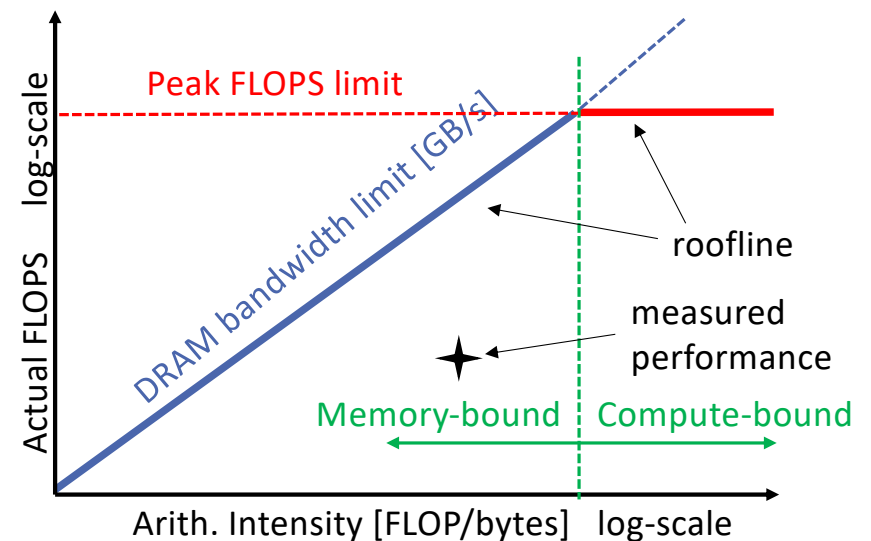
<https://crd.lbl.gov/departments/computer-science/PAR/research/roofline>

Arithmetic Intensity

- The ratio between the number of executed operations and the number of bytes transferred between the CPU and the memory is called arithmetic intensity.
- Smaller arithmetic intensity means a larger pressure on the memory subsystem, and conversely, larger arithmetic intensity means a larger pressure on the CPUs computational resources.

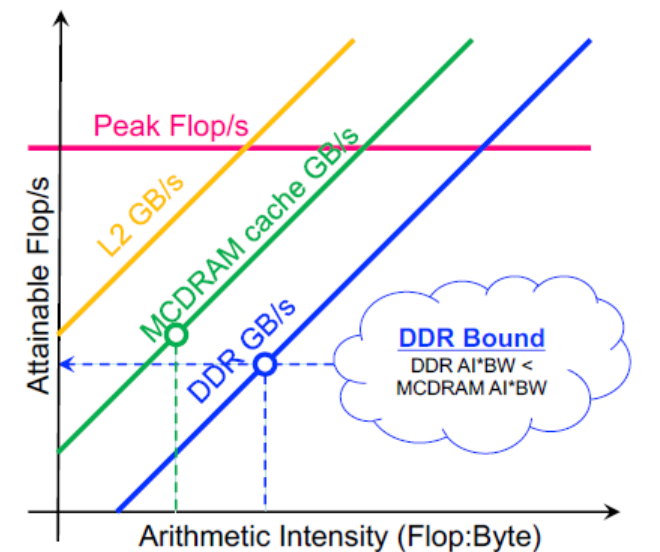
Naïve Roofline Performance Model

- Assumptions: Considers DRAM and Processor
- Actual FLOPS are limited first by DRAM bandwidth (memory-bound) and then by CPU (or GPU) Peak FLOPS (compute-bound)
- Actual measured performance is below the roofline
- Depending on Arithmetic Intensity:
 - **memory-bound** code
 - **compute-bound** code
- **Log-log scale is used for clarity**



More complex Roofline Models

- Assumptions: Considers also the Cashes
- We considered a basic **DRAM-only Roofline Model**:
 - Bytes as seen from DRAM access
- Not covered: Hierarchical Roof-line
 - for problems that fit in the cache we can add:
 - L1, L2, L3 bandwidth
 - Each Cache Level has its own A.I. (different bytes going through that level of the mem. Hierarchy)
- Also not covered: Cache-aware Roof-line
 - FLOP/bytes as seen from CORE
 - Different roof but same A.I.
 - Need to know from which level data is coming
 - <http://www.inesc-id.pt/ficheiros/publicacoes/9068.pdf>



Hierarchical Roofline
from: LBNL (SC17 Roofline Model Workshop slides)

A computer node at Columbia

- Intel Xeon E5630@2.53GHz performance:
 1. #cores: 4
 2. LLC (L3) size: 12MB
 3. Clock frequency: 2.53GHz
 4. **DRAM peak bandwidth:** 25.6 GB/s
 5. **CPU Peak FLOPS:** 81.3 DP GFLOPS – 162.56 SP GFLOPS
- DRAM peak bandwidth:
 - https://ark.intel.com/products/47924/Intel-Xeon-Processor-E5630-12M-Cache-2_53-GHz-5_86-GTs-Intel-QPI
- CPU peak FLOPS :
 - $\text{FLOPS} = \text{frequency} * \text{total_cores} * \text{FLOPS/cyc}$
 - <https://en.wikipedia.org/wiki/FLOPS> (architectures list - this is a *Sandy Bridge*)

```
$ ssh username@...
```

```
$ ssh node..
```

```
$ cat /proc/cpuinfo
```

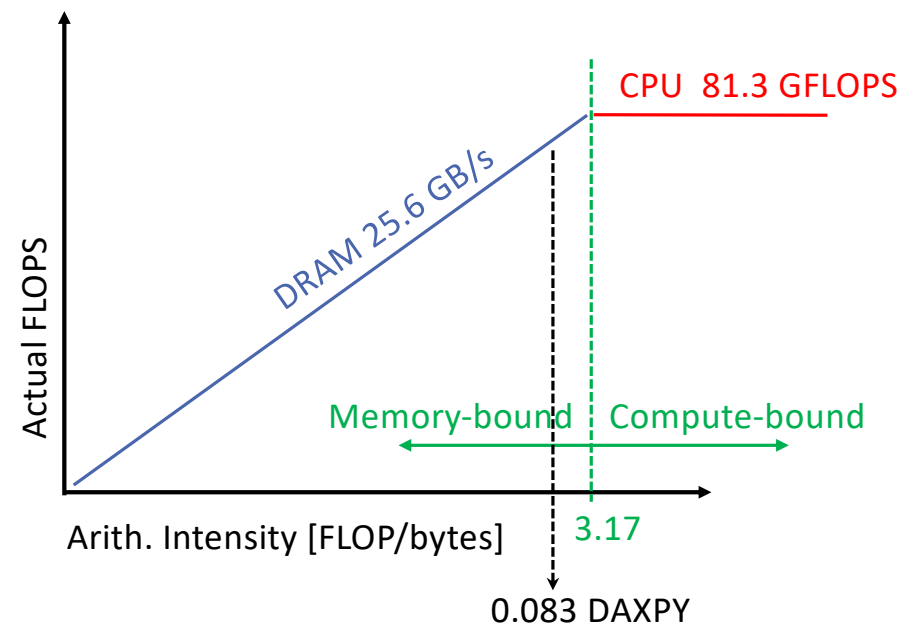
```
processor      : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 44
model name    : Intel(R) Xeon(R) CPU E5630 @ 2.53GHz
stepping      : 2
microcode     : 0x15
cpu MHz       : 2527.014
cache size    : 12288 KB
physical id   : 0
siblings      : 8
core id       : 10
cpu cores     : 4
```

Roofline Model Example – Columbia Compute Node

- *crackle1*:
 - CPU peak: 81.3 DP GFLOPS
 - DRAM peak BW: 25.6 GB/s
- DAXPY code (**Double-precision A·X Plus Y.**):

```
for (i=0;i<N;i++) {  
  Z[i]= A * (X[i] + Y[i])  
}
```

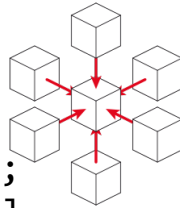
- Y,A,X are 64-bit float (DP)
- DRAM and CPU cross at:
 - $81.3 \text{ GFLOPS} / 25.6 \text{ GB/s} = 3.17 \text{ FLOP/byte}$
 - CPU_peak/DRAM_BW
 - Where DP-bytes=8, SP-bytes=4, HP-bytes=2
- A.I. = $2 \text{ FLOP} / (3 \cdot 8) \text{ bytes} = 0.083 \text{ FLOP/byte}$
- Result: $0.083 < 3.17 \Rightarrow$ **Memory-bound** $\Rightarrow$ how far for DRAM BW?



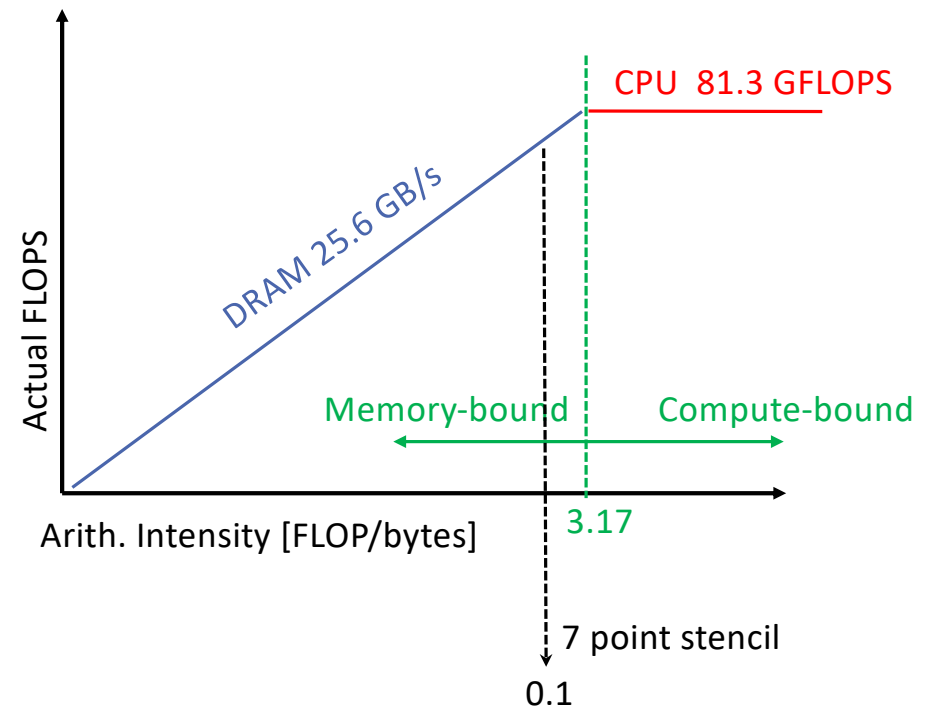
Roofline Model Example (2) – Columbia Compute Node

- 7-Points Stencil code:

```
for(k=1;k<N;k++){  
  for(j=1;j<N;j++){  
    for(i=1;i<N;i++){  
      int ijk = i+j*jStride+k*kStride;  
      new[ijk] = -6.0*old[ijk  
        +old[ijk-1  
        +old[ijk+1  
        +old[ijk-jStride]  
        +old[ijk+jStride]  
        +old[ijk-kStride]  
        +old[ijk+kStride];}}}
```

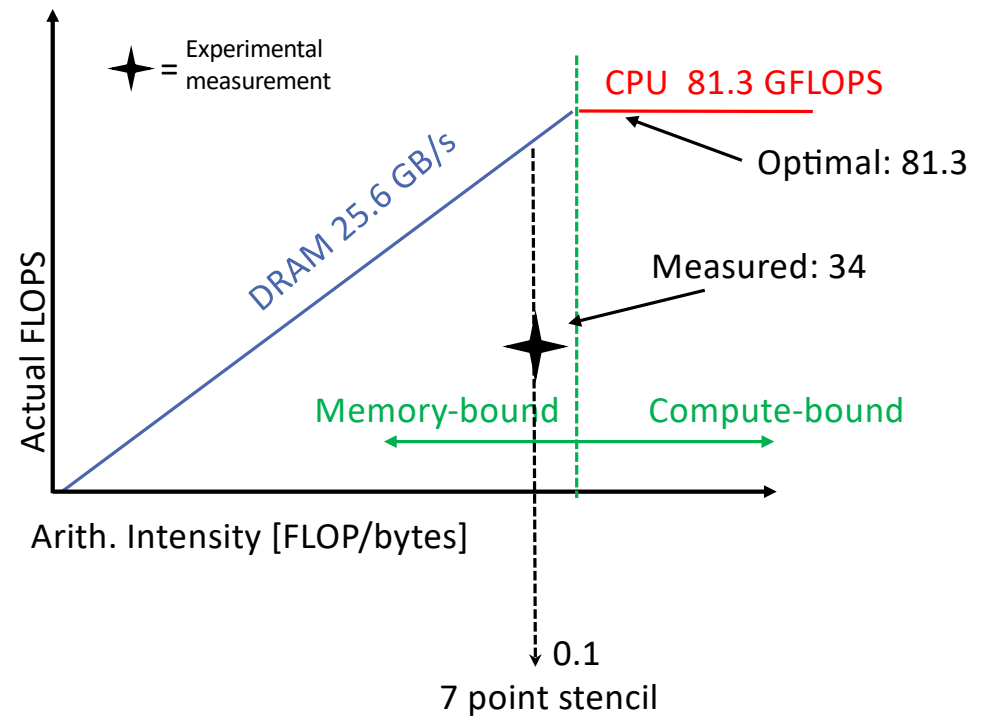


- 7 DP flops (new[] is 64bits)
- 8 memory references
- DRAM/CPU cross = 3.17 FLOP/byte
- AI = 7 FLOP/(8*8) bytes = 0.109 FLOP/byte
- Result: $0.109 < 3.17 \Rightarrow$ still **Memory Bound** $\Rightarrow$ how to optimize?



Next steps - Optimization

1. Know the limitation from the model: CPU vs. DRAM
2. Measure actual performance:
Example: 34 GFLOPS
3. Optimize to get close to max FLOPS! (81.3 GFLOPS)



Performance optimization methodology (3): Optimization

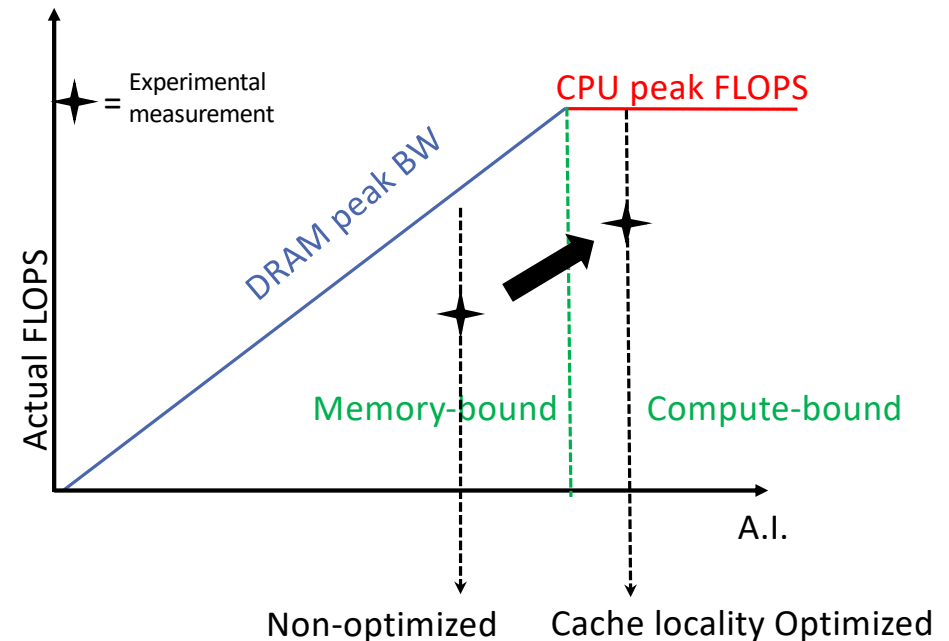
Two ways to Performance

- Reduce latency (do one operation faster):
 - Data access latency reduction
- Increase Parallelism (do more operations at the same time):
 - Vectorization
 - Instruction Level Parallelism
 - Thread Level Parallelism
 - Multi-core design
 - Computer Clusters

Optimization Example: Cache Blocking

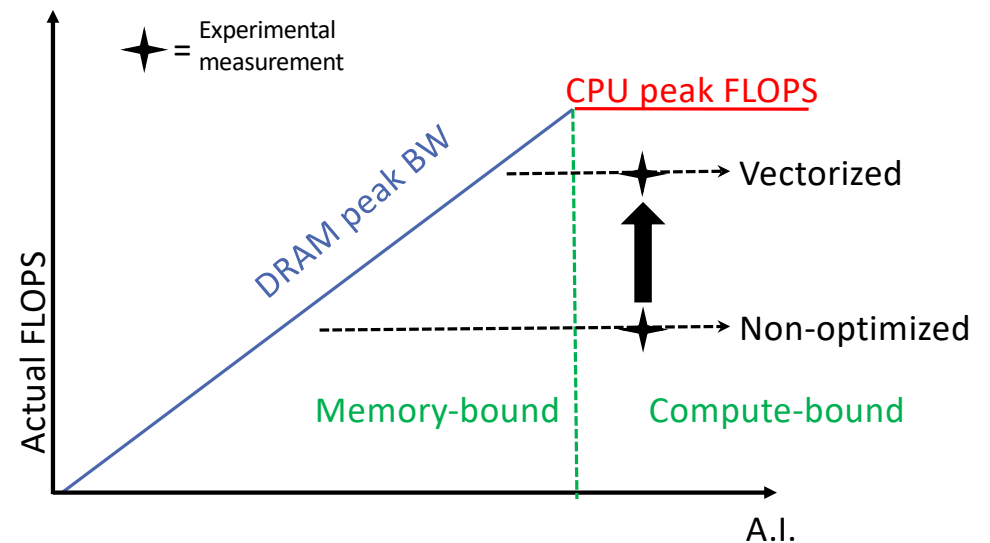
- Observation: Cache access latency is about 10x lower than DRAM and BW is much higher
- Optimization (cache blocking):
 - Divide program data structures in blocks of the **cache size**
 - Work on each block before switching to the next
 - Less DRAM bytes: **cache is filtering DRAM accesses**
 - A.I. [FLOPS/(DRAM bytes)] is higher
- Result:
 - Bottleneck moves: Code (may) become compute-bound with higher FLOPS!

<https://www.intel.com/content/www/us/en/developer/articles/technical/cache-blocking-techniques.html>



Optimization Example: Vectorization

- Observation: SIMD instructions execute multiple FLOP (2,4,8,16..) with 1 instruction => higher FLOPS
- Optimization (Vectorization):
 - Replace normal code with SIMD instructions
 - **Hint:** use math libraries like BLAS (CPU) or cuDNN (GPU) and they will do it for you!
- Result:
 - Code reaches higher FLOPS!



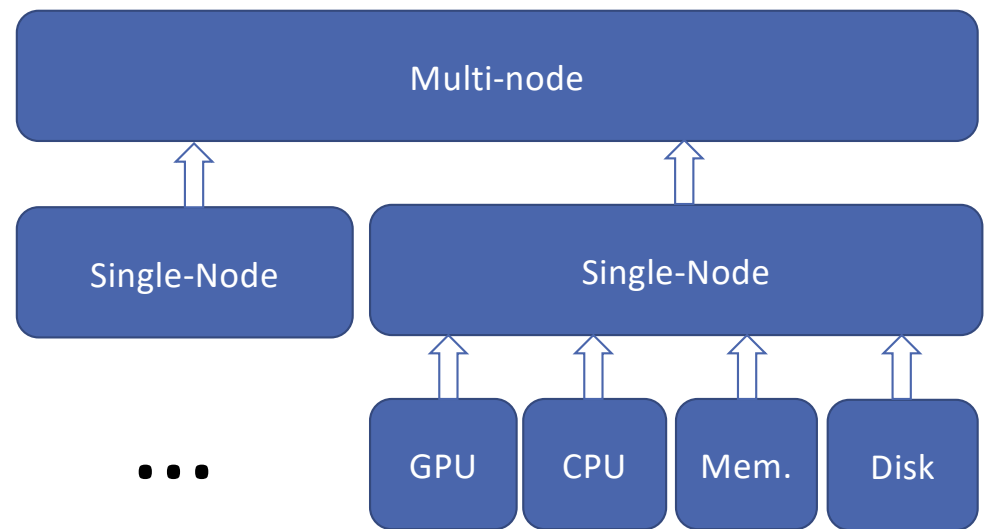
Hierarchical Perf. Optimization – (next lessons)

- Single Node optimizations:

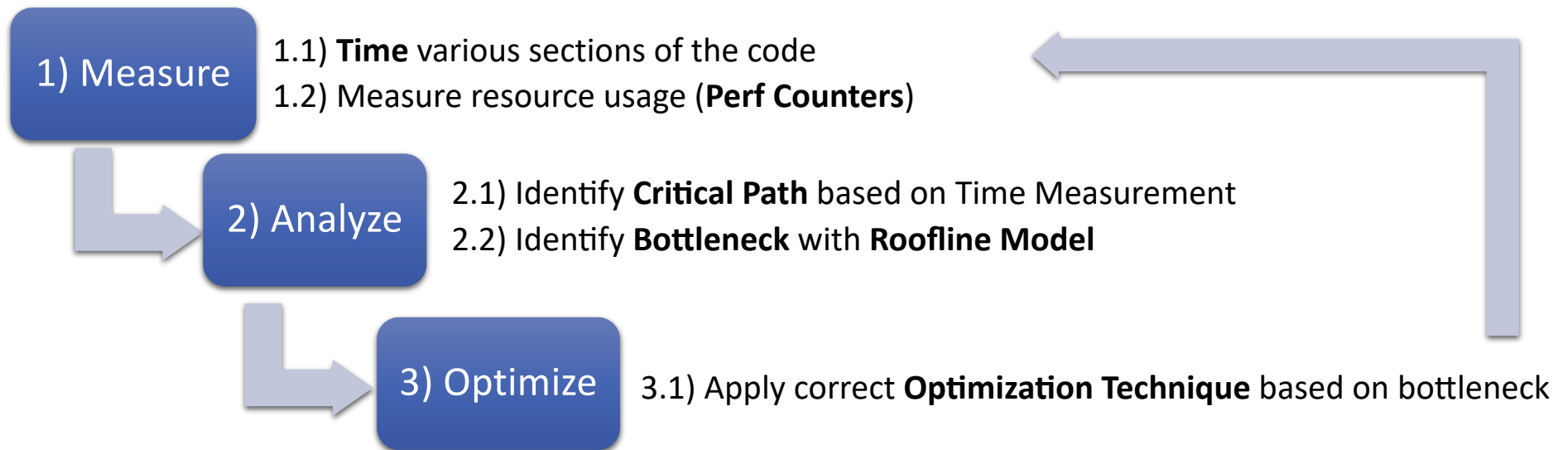
- CPU:
 - Vectorize/SIMD optimizations
 - CPU Cache/Memory optimizations
 - Multi-core scalability/parallelism
- GPU:
 - SM Optimizations
 - SM Cache/Memory optimization
- Disk and IO

- Multi-node optimizations:

- Parallelism exposure
- Domain decomposition
- Load-balancing
- Reduce synchronizations
- Reduce collectives



Performance Optimization Methodology Recap



Introduction to High Performance Machine Learning

Lecture 2 (continued) 02/09/26

Dr. Kaoutar El Maghraoui

Gradient Descent Optimization Algorithms and PyTorch

ML Performance Factors

Algorithm Performance

- Training vs. Inference Performance
- Distributed Training Algorithms. Ex: Asynchronous vs. Synchronous Distributed SGD
- Inference-Centric Algorithms (Prefill vs. Decode, Speculative Decoding)
- KV-Cache Optimization & attention variants
- Algorithms for Efficient Inference. Ex: Quantization and Pruning

Hyperparameters Performance

- Learning Rate & Schedulers, Network Depth & Width, Batch Size & Microbatch Size, Context Length / Sequence Length, KV-Cache Precision, Inference Sampling Parameters (Temperature, Top-k, Top-p)

Implementation Performance

- Algorithms Implementation (vectorization, comm./comp. overlapping, parallelization, data access)

Framework Performance

- ML Frameworks: PyTorch, TensorFlow, Caffe, MXNET.
- Graph compilers and Distributed Runtimes
- Inference serving frameworks

Library Performance

- Math libraries (cuDNN), Communication Libraries (MPI, NCCL, GLU), Transformer libraries

Hardware Performance

- CPU, DRAM, GPU, HBM, Storage, Tensor Cores/Matrix Engines, Disk/Filesystem, Network/Interconnect

System view:
Bottom-Up
(this course)

Algorithmic view:
Top-down

ML Performance Factors

Algorithm Performance

- Training vs. Inference Performance
- Distributed Training Algorithms. Ex: Asynchronous vs. Synchronous Distributed SGD
- Inference-Centric Algorithms (Prefill vs. Decode, Speculative Decoding)
- KV-Cache Optimization & attention variants
- Algorithms for Efficient Inference. Ex: Quantization and Pruning

Hyperparameters Performance

- Learning Rate & Schedulers, Network Depth & Width, Batch Size & Microbatch Size, Context Length / Sequence Length, KV-Cache Precision, Inference Sampling Parameters (Temperature, Top-k, Top-p)

Implementation Performance

- Algorithms Implementation (vectorization, comm./comp. overlapping, parallelization, data access)

Framework Performance

- ML Frameworks: PyTorch, TensorFlow, Caffe, MXNET.
- Graph compilers and Distributed Runtimes
- Inference serving frameworks

Library Performance

- Math libraries (cuDNN), Communication Libraries (MPI, NCCL, GLU), Transformer libraries

Hardware Performance

- CPU, DRAM, GPU, HBM, Storage, Tensor Cores/Matrix Engines, Disk/Filesystem, Network/Interconnect

Hyperparameters Performance

- **Learning Rate & Schedulers:**
Cosine decay, warmup, and adaptive schedules strongly affect convergence speed and stability.
- **Network Dimensions:**
Depth and width determine model capacity but directly increase memory and compute cost.
- **Batch Size & Microbatch Size:**
Larger batches improve hardware utilization but may hurt convergence or exceed memory limits.
- **Context Length:**
Longer sequences significantly increase attention cost and KV-cache size.
- **KV-Cache Precision:**
Choosing FP16, FP8, or INT8 trades accuracy for memory footprint and bandwidth.
- **Inference Sampling Parameters:**
Temperature, top-k, and top-p influence output quality and sometimes runtime behavior.

The Deep Learning Lifecycle - Recap

Phase 01

Training

Discovering **weights and biases** via iterative optimization.

Requires **Backpropagation** to compute gradients for every layer.

High memory overhead due to storing **intermediate activations**.

Phase 02

Inference

Using **frozen parameters** to make real-world predictions.

Executes only the **Forward Pass**; no gradients are computed.

Optimized for **latency and throughput** in production environments.

Computational Cost Breakdown

- The **Backward Pass** typically requires **~2x** the computational cost of the Forward Pass due to gradient accumulation.
- **Modern Reality:** In production LLM systems, inference dominates total compute cost and system complexity.

Batch Size & Utilization

Efficiency Low

Small Batch Size

- **Noisy Updates:**
High variance in gradient estimates.
- **Memory Bound:**
Low arithmetic intensity; GPU stalls waiting for data.
- **Under-utilization:**
Fails to saturate Tensor Core throughput.

The Generalization Caveat

While large batches maximize hardware efficiency, **extremely large batches** can lead to a "generalization gap" and require careful learning rate scaling (e.g., Linear Scaling Rule).

Efficiency High

Large Batch Size

- **Stable Updates:**
More accurate gradient estimation.
- **Compute Bound:**
High arithmetic intensity; maximizes GFLOPS.
- **High Throughput:**
Better utilization of HBM bandwidth and cores.

Modern LLM Nuance

- Training uses large global batch sizes with gradient accumulation.
- Inference uses dynamic batching to amortize KV-cache and kernel launch overhead.
- Optimal batch size is workload- and latency-SLO dependent.

Gradient Descent Mechanics - recap

The Update Rule

$$\theta = \theta - \alpha \nabla L(\theta)$$

Where θ is the parameters vector, α is the learning rate

and $\nabla J(\theta)$ is the gradient of the cost

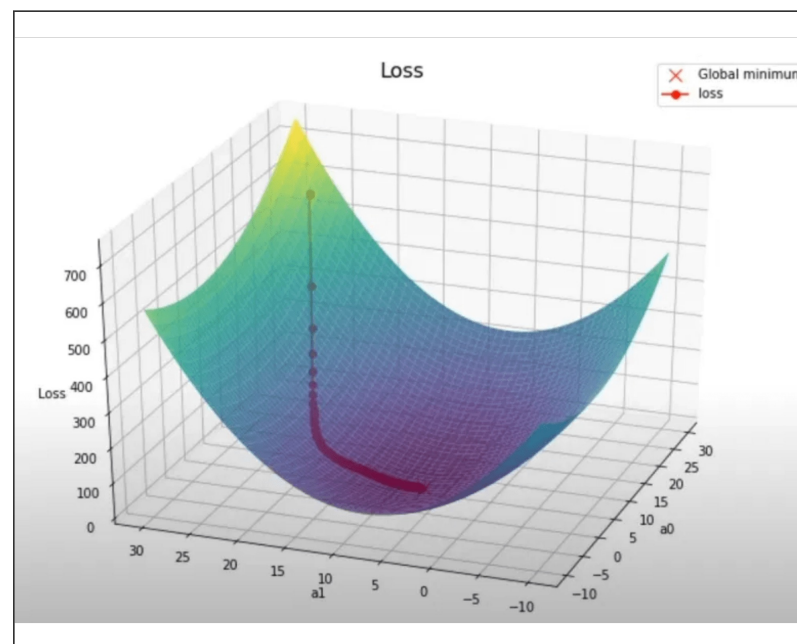
Optimization Variants

Stochastic (SGD)

Update parameters for **each training sample**. High variance in updates, can help escape local minima.

Mini-batch

Update parameters for a **group of samples**. Standard for GPU utilization and stable convergence.



Visualization of a 3D loss surface showing the optimization path towards the global minimum.

EMA: Smoothing Gradients

Exponential Moving Average Formula

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$

The Intuition

- v_t : the current EMA value.
- θ_t : the new data point at time t .
- β : smoothing factor ($0 < \beta < 1$).

The β parameter controls the "memory" of the average. It effectively averages over the last $1 / (1 - \beta)$ steps. High β (0.99) means long memory and smooth updates; low β (0.9) means short memory and more reactive updates.

The Impact

Why Smooth?

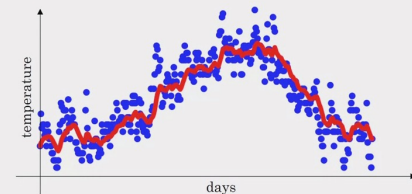
Gradients in deep learning are inherently **noisy** due to mini-batch sampling. EMA dampens high-frequency oscillations while preserving the low-frequency trend of the descent.

Core Applications

EMA is the mathematical engine behind **Momentum**, **RMSProp**, and **Adam**. It allows optimizers to navigate complex loss surfaces with stability.

Exponentially weighted averages

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t$$



Optimizers: Optimizing for Time to Convergence

The Core Performance Metric

- In High-Performance ML, we care about **Time to Accuracy**, not just **Time per Epoch**. Optimization must balance hardware throughput with algorithmic convergence speed.
- In large-scale LLM training, **Time to Target Loss** is more important than per-epoch speed.

Momentum

Smooths noisy gradients using an **Exponential Moving Average (EMA)** of past updates.

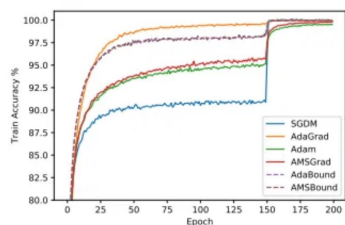
Accelerates descent in relevant directions and dampens oscillations in steep, narrow valleys.

Nesterov Momentum

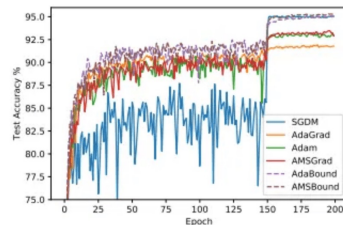
A "**Look-ahead**" variant that computes the gradient at the predicted next position.

Provides better stability and prevents overshooting, leading to faster convergence in practice.

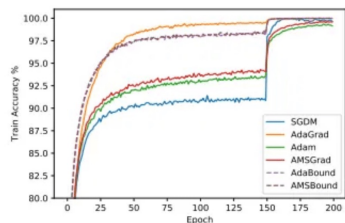
Adaptive Methods: RMSProp & Adam



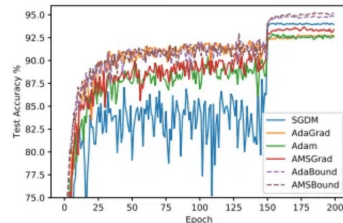
(a) Training Accuracy for DenseNet-121



(b) Test Accuracy for DenseNet-121



(c) Training Accuracy for ResNet-34



(d) Test Accuracy for ResNet-34

RMSProp

Scales the learning rate for each parameter by a running average of the **magnitudes of recent gradients** (squared gradients).

Adam

Combines **Momentum** (1st moment) and **RMSProp** (2nd moment). It adapts learning rates while maintaining a stable update direction.

The Industry Standard

Adam is the go-to optimizer for most deep learning tasks due to its robustness and minimal hyperparameter tuning requirements.

Convergence comparison of various optimizers on deep networks

Modern Standard: AdamW

The Problem

Coupled Weight Decay

In the original Adam implementation, **L2 regularization** was coupled with the gradient updates. This caused the weight decay to be scaled by the adaptive learning rate, leading to sub-optimal regularization.

Modern Usage

AdamW is the **de facto standard** for training Large Language Models (LLMs) like **Llama 3, GPT-4, and DeepSeek** due to its superior generalization properties.

The Solution

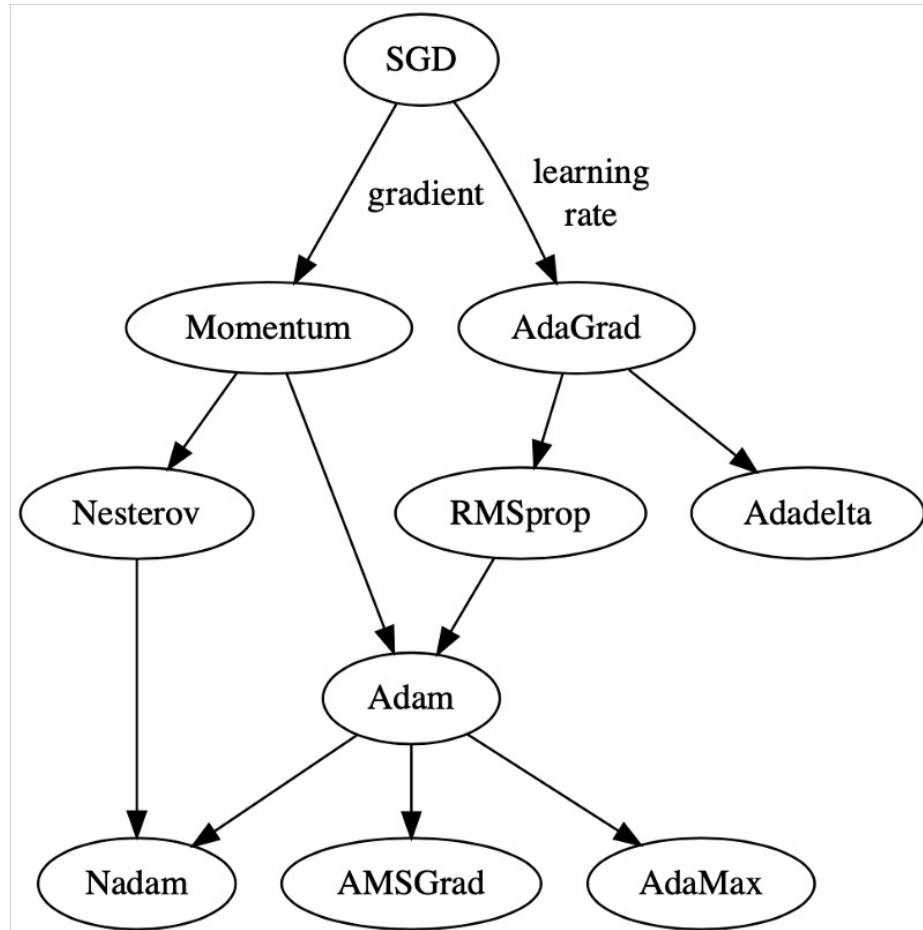
Decoupled Weight Decay

AdamW **decouples** the weight decay from the gradient update step. The decay is applied directly to the weights, independent of the adaptive moments.

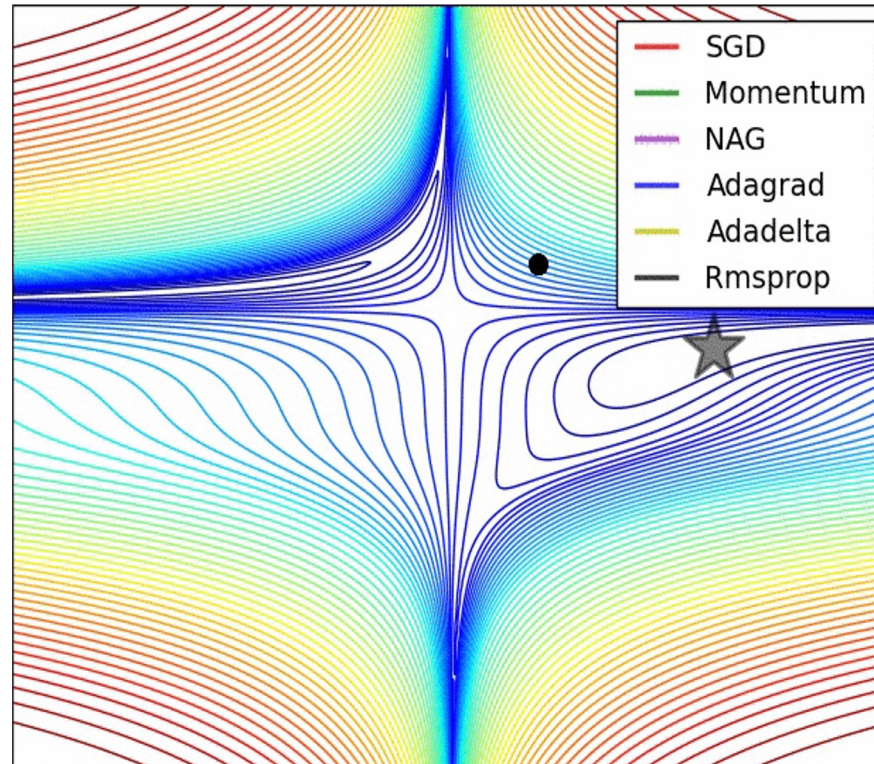
Key Advantages

- Restores the original intent of weight decay.
- Improves **generalization** on complex datasets.
- Simplifies hyperparameter tuning for large-scale models.

Evolutionary Map of Optimizers

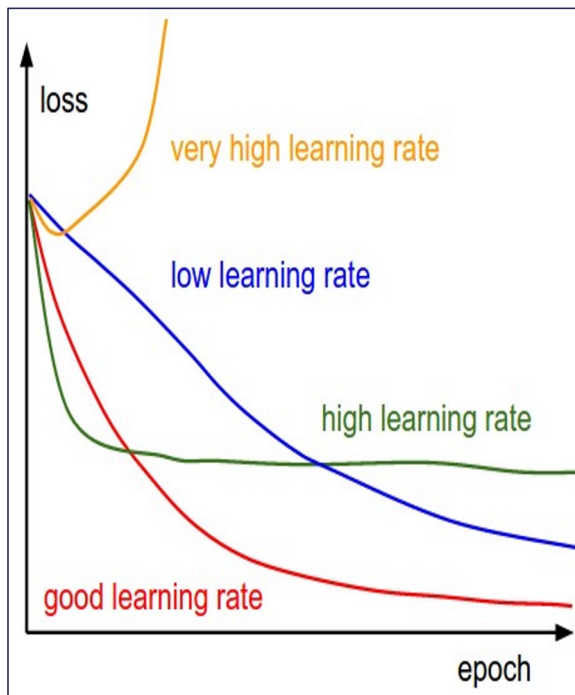


The effects of different update formulas



(image credits to Alec Radford)

SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



=> Learning rate decay over time!

step decay:

e.g. decay learning rate by half every few epochs.

exponential decay:

$$\alpha = \alpha_0 e^{-kt}$$

1/t decay:

$$\alpha = \alpha_0 / (1 + kt)$$

Optimizers Cheat Sheet

Notations

- t —time step
- w —weight/parameter which we want to update
- α —learning rate
- $\partial L / \partial w$ —gradient of L , the loss function to minimise, w.r.t. to w

Vanilla SGD

$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

Momentum

$$w_{t+1} = w_t - \alpha V_t$$

$$V_t = \beta V_{t-1} + (1 - \beta) \frac{\partial L}{\partial w_t}$$

Nesterov

$$w_{t+1} = w_t - \alpha V_t$$

$$V_t = \beta V_{t-1} + (1 - \beta) \frac{\partial L}{\partial w^*}$$

$$w^* = w_t - \alpha V_{t-1}$$

Adagrad

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{S_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$S_t = S_{t-1} + \left[\frac{\partial L}{\partial w_t} \right]^2$$

RMSprop

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{S_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$S_t = \beta S_{t-1} + (1 - \beta) \left[\frac{\partial L}{\partial w_t} \right]^2$$

Adadelta

$$w_{t+1} = w_t - \frac{\sqrt{D_{t-1} + \epsilon}}{\sqrt{S_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$D_t = \beta D_{t-1} + (1 - \beta) [\Delta w_t]^2$$

$$S_t = \beta S_{t-1} + (1 - \beta) \left[\frac{\partial L}{\partial w_t} \right]^2$$

Adam

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{S}_t + \epsilon}} \cdot \hat{V}_t$$

$$\hat{V}_t = \frac{V_t}{1 - \beta_1^t}$$

$$\hat{S}_t = \frac{S_t}{1 - \beta_2^t}$$

$$V_t = \beta_1 V_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

$$S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left[\frac{\partial L}{\partial w_t} \right]^2$$

AdaMax

$$w_{t+1} = w_t - \frac{\alpha}{\hat{S}_t} \cdot \hat{V}_t$$

$$\hat{V}_t = \frac{V_t}{1 - \beta_1^t}$$

$$V_t = \beta_1 V_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

$$S_t = \max(\beta_2 S_{t-1}, \left| \frac{\partial L}{\partial w_t} \right|)$$

Nadam

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{S}_t + \epsilon}} \left(\beta_1 \hat{V}_{t-1} + \frac{1 - \beta_1}{1 - \beta_1^t} \cdot \frac{\partial L}{\partial w_t} \right)$$

$$\hat{V}_t = \frac{V_t}{1 - \beta_1^t}$$

$$\hat{S}_t = \frac{S_t}{1 - \beta_2^t}$$

$$V_t = \beta_1 V_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

$$S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left[\frac{\partial L}{\partial w_t} \right]^2$$

AMSGrad

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{S}_t + \epsilon}} \cdot V_t$$

$$\hat{S}_t = \max(\hat{S}_{t-1}, S_t)$$

$$V_t = \beta_1 V_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

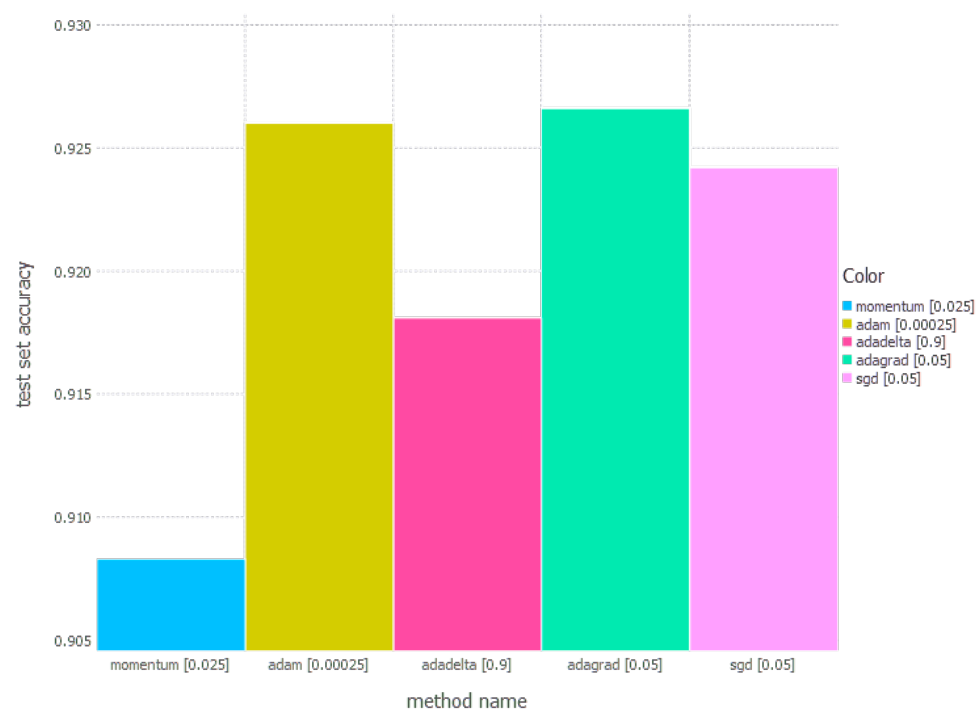
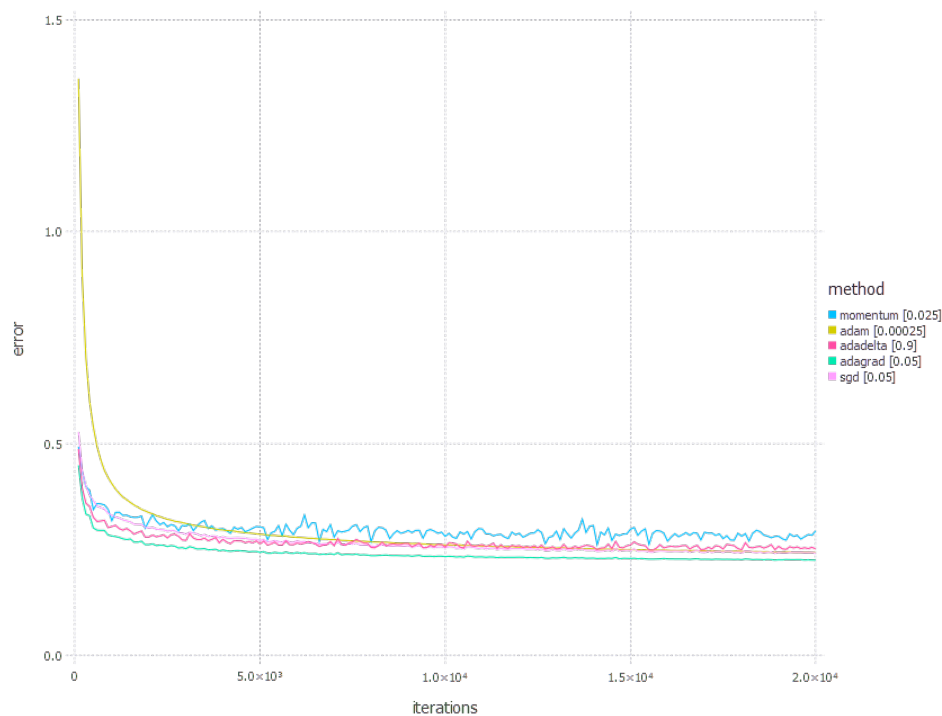
$$S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left[\frac{\partial L}{\partial w_t} \right]^2$$

Optimization Techniques Comparison (I)

- Toy example on MNIST Classification
- Three different network architecture tested:
 1. network with linear layer and softmax output (softmax classification)
 2. network with sigmoid layer (100 neurons), linear layer and softmax output
 3. network with sigmoid layer (300 neurons), ReLU layer (100 neurons), sigmoid layer (50 neurons) again, linear layer and softmax output
- Mini-batch size of 128
- Run the algorithm for approx. 42 epochs (20000 iterations)
- <http://int8.io/comparison-of-optimization-techniques-stochastic-gradient-descent-momentum-adagrad-and-adadelta/>

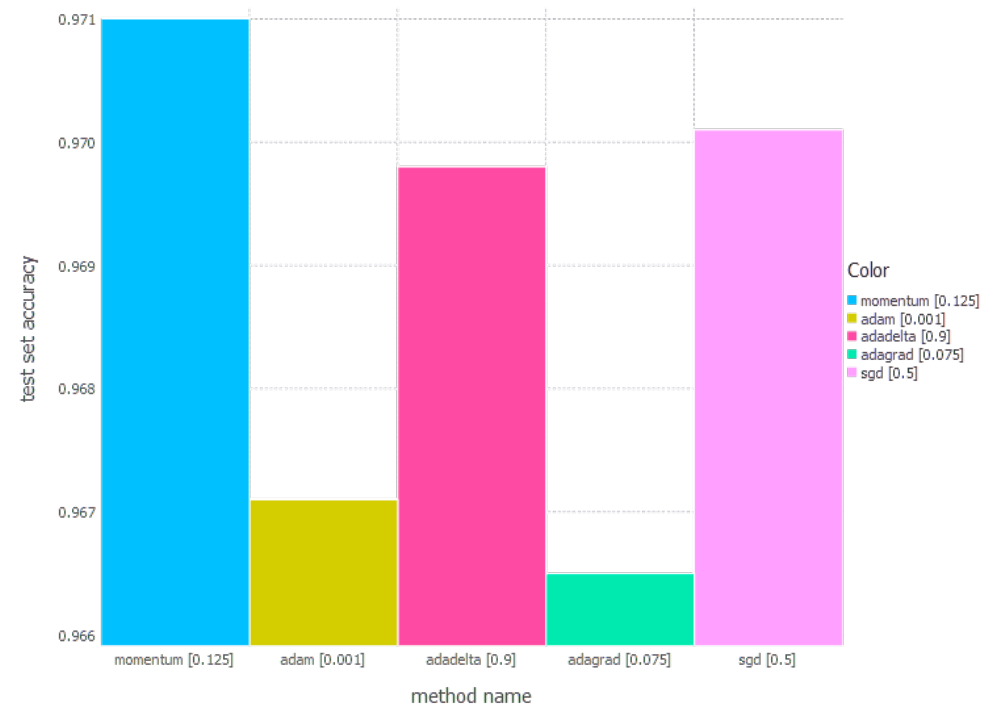
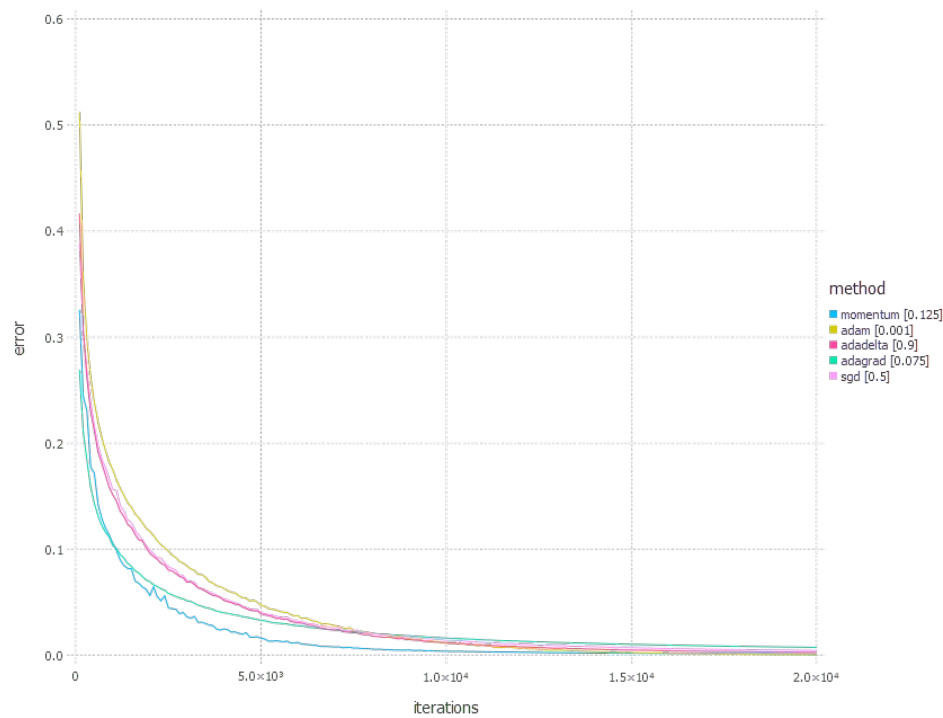
Results on net 1

network with linear layer and softmax output (softmax classification)



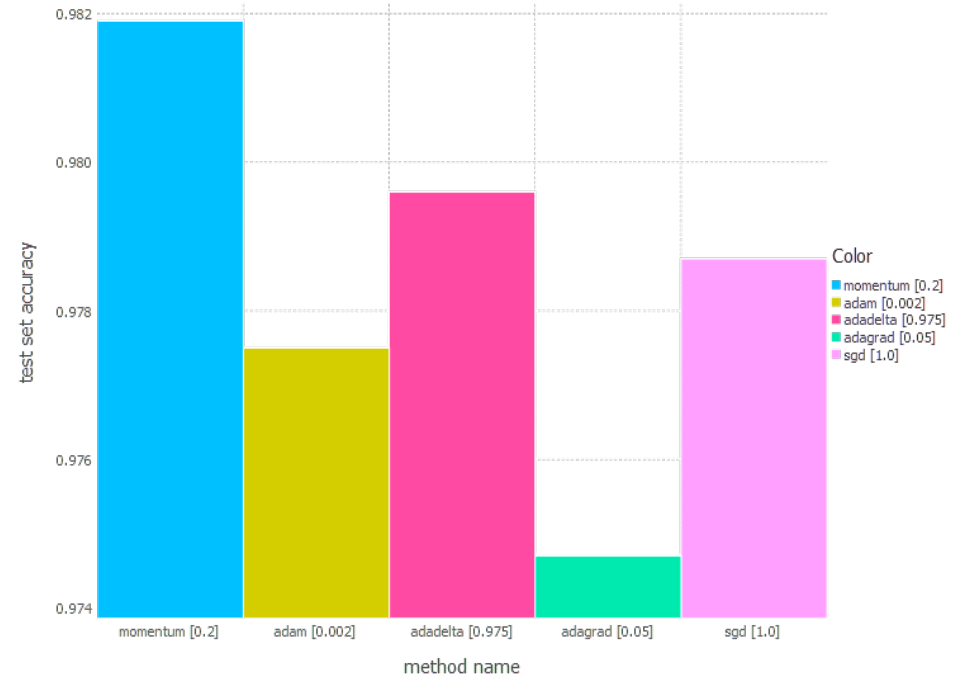
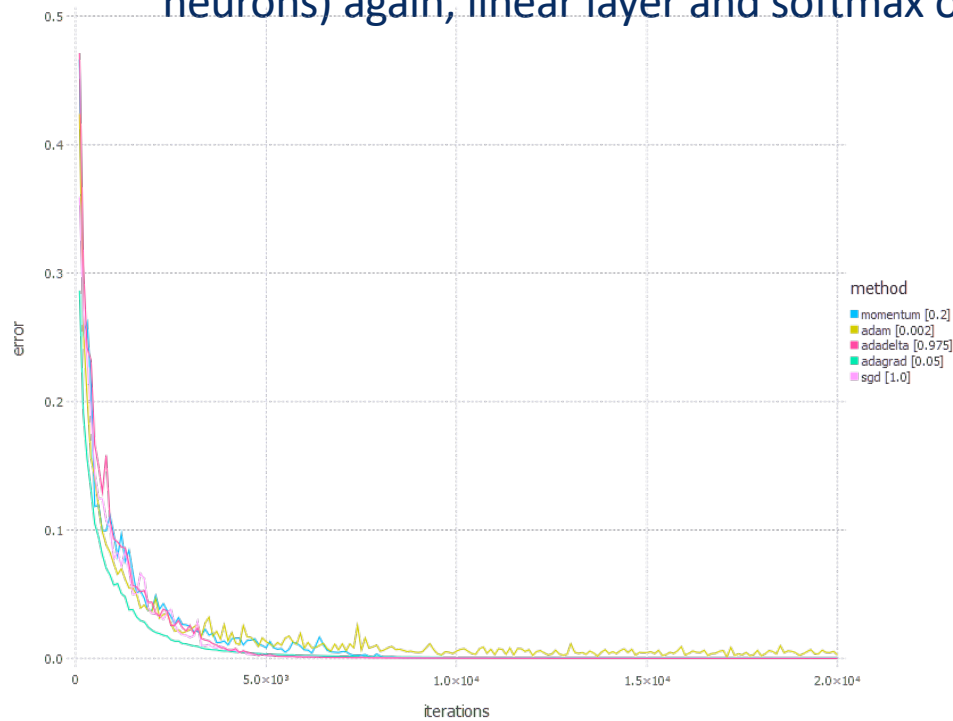
Results on net 2

network with sigmoid layer (100 neurons), linear layer and softmax output



Results on net 3

network with sigmoid layer (300 neurons), ReLU layer (100 neurons), sigmoid layer (50 neurons) again, linear layer and softmax output



So....Which optimizer to use?

- There is no universally optimal optimizer.
- Optimizer choice depends on:
 - Model scale
 - Memory footprint
 - Training stability
- In practice:
 - AdamW → default for LLM pretraining and fine-tuning
 - Lion → memory-efficient alternative at scale
 - SGD → rarely used for LLMs, still relevant for vision

Comments on optimizer

- All the optimizers take in the parameters and gradients and manipulate over them.
- None of the optimizers are theoretically proven to have better convergence rate than SGD, except Nesterov momentum.
- Even Nesterov momentum is only proved to work on strongly convex problems.
- In practice, SGD with momentum and a per-iteration learning rate schedule (poly, cosine) often produce better final accuracy with some tuning than automatic update algorithms.

Optimizers in Large Language Models

- Large language models shift optimizer design from pure convergence speed to **stability, memory efficiency, and scalability**.
- Large language models require specialized optimizers that efficiently handle large-scale training, often balancing convergence speed, stability, and memory efficiency.
- What optimizers were used to train **LLaMA, GPT, and DeepSeek?**

LLaMA (Large Language Model Meta AI)

- **Optimizer Used: AdamW**
 - Meta used **AdamW** (Adam with decoupled weight decay) to train LLaMA models.
- **Why AdamW?**
 - **Weight decay decoupling:** Unlike traditional Adam, AdamW applies weight decay **separately from gradient updates**, which improves generalization.
 - **Memory efficiency:** While AdamW still stores first and second moments (like Adam), it's more stable than plain Adam in large-scale training.
 - **Improved convergence:** Helps mitigate overfitting and makes training stable, especially with large batch sizes.
- **Reference:**
 - LLaMA models (7B, 13B, 65B) were trained using AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, and a **learning rate of 3×10^{-4} with cosine decay**.



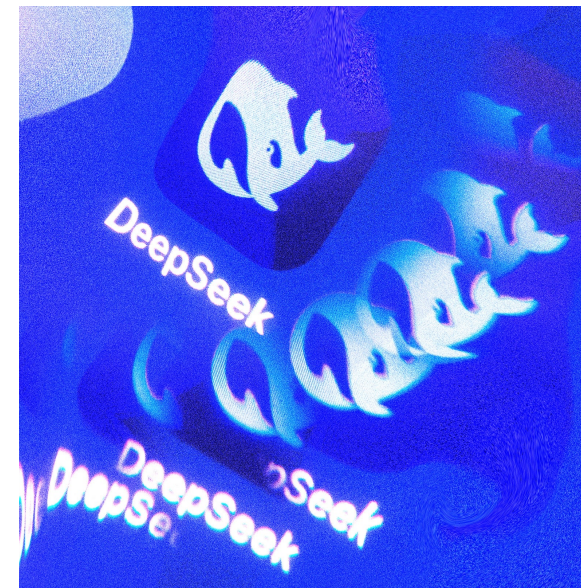
GPT (OpenAI GPT-3, GPT-4)

- **Optimizer Used: Adam with custom modifications**
 - OpenAI uses **Adam** but with a **custom learning rate schedule** and optimizations tailored for training at extreme scales.
- **Why Adam?**
 - **Adaptive learning rate scaling:** Helps in dynamically adjusting step sizes across different parameters, which is crucial for stability in deep networks.
 - **Variance control:** Uses $\beta_1 = 0.9$, $\beta_2 = 0.98$ (or 0.95) for improved stability.
 - **Training speed:** Adam performs well with large batch sizes and mixed-precision training (FP16/BF16), which accelerates model convergence.
 - **Custom modifications:** OpenAI likely introduces additional tweaks such as:
 - **Gradient clipping** to prevent exploding gradients.
 - **Learning rate warmup** for stable training at the beginning.
- **Reference:**
 - GPT-3 was trained using Adam with a **batch size of 3.2M tokens per step**, learning rate 6×10^{-4} , and **400B tokens**.



DeepSeek (DeepSeek-7B, DeepSeek-67B)

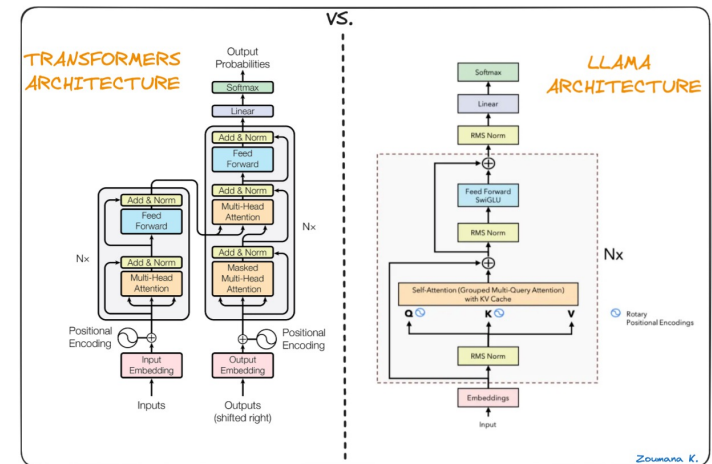
- **Optimizer Used: Lion (EvoLved Sign Momentum)**
 - DeepSeek models use **Lion** (EvoLved Sign Momentum), a more efficient alternative to Adam.
- **Why Lion?**
 - **Lower memory usage:** Unlike Adam, which tracks both first and second moments, Lion **only tracks momentum**, significantly reducing memory overhead.
 - **Faster convergence:** Lion updates parameters using the **sign of the gradient**, which can lead to more robust generalization.
 - **Better training stability:** Works well with **larger batch sizes** while avoiding overfitting.
- **Reference:**
 - DeepSeek-67B was trained using **Lion with $\beta_1 = 0.95$, $\beta_2 = 0.98$, a learning rate of 2×10^{-3} , and 50B tokens.**



<https://medium.com/@yash9439/lion-optimizer-73d3fd18abe9>

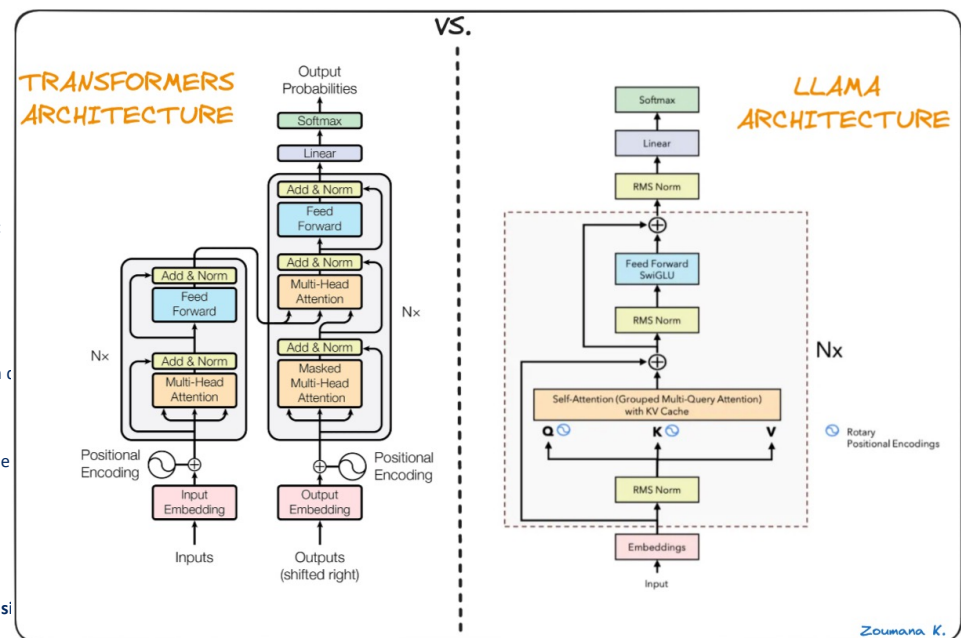
Example: Hyperparameters in the LLaMA Model

- **Initial Variants:** LLaMA 1 was released with 4 different model variants:
 - 6.7B parameters
 - 13B parameters
 - 32.5B parameters
 - 65.2B parameters
- **Multi-Head Attention:**
 - The number of attention heads in LLaMA is significantly higher than in the original transformer model:
 - 6.7B model: 32 heads
 - 13B model: 40 heads
 - 32.5B model: 52 heads
 - 65.2B model: 64 heads
 - LLaMA uses a **modified version of multi-head attention**, called **grouped multi-query attention**, which differs from the traditional transformer's multi-head attention (which typically had 8 heads).
- **Token Embedding Dimensionality:**
 - The input tokens in LLaMA models are represented by vectors of varying dimensions, depending on the model size:
 - 6.7B model: 4096-dimensional vectors
 - 13B model: 5120-dimensional vectors
 - 32.5B model: 6656-dimensional vectors
 - 65.2B model: 8192-dimensional vectors
 - This is a significant increase compared to the original transformer architecture, which used **512-dimensional vectors**.
- **Learned Input Embeddings:**
 - Unlike traditional models with static embeddings, **LLaMA uses learned input embeddings** that are trained during the training process itself.



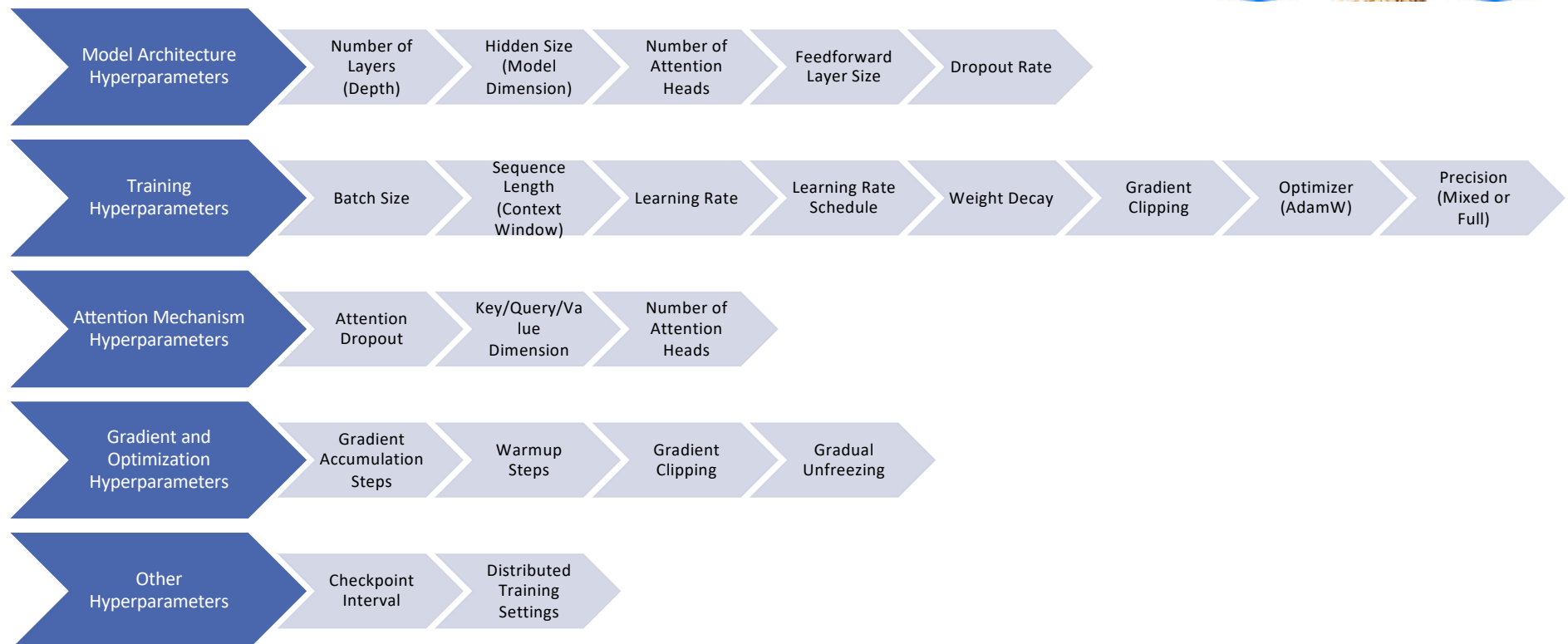
Example: Hyperparameters in the LLaMA Model

- **Initial Variants:** LLaMA 1 was released with 4 different model variants:
 - 6.7B parameters
 - 13B parameters
 - 32.5B parameters
 - 65.2B parameters
- **Multi-Head Attention:**
 - The number of attention heads in LLaMA is significantly higher than in the original transformer model:
 - 6.7B model: 32 heads
 - 13B model: 40 heads
 - 32.5B model: 52 heads
 - 65.2B model: 64 heads
 - LLaMA uses a **modified version of multi-head attention**, called **grouped multi-query attention**, which differs from the traditional transformer's multi-head attention (which typically had 8 heads).
- **Token Embedding Dimensionality:**
 - The input tokens in LLaMA models are represented by vectors of varying dimensions, depending on the model size:
 - 6.7B model: 4096-dimensional vectors
 - 13B model: 5120-dimensional vectors
 - 32.5B model: 6656-dimensional vectors
 - 65.2B model: 8192-dimensional vectors
 - This is a significant increase compared to the original transformer architecture, which used **512-dimensions**.
- **Learned Input Embeddings:**
 - Unlike traditional models with static embeddings, **LLaMA uses learned input embeddings** that are trained during the training process itself.



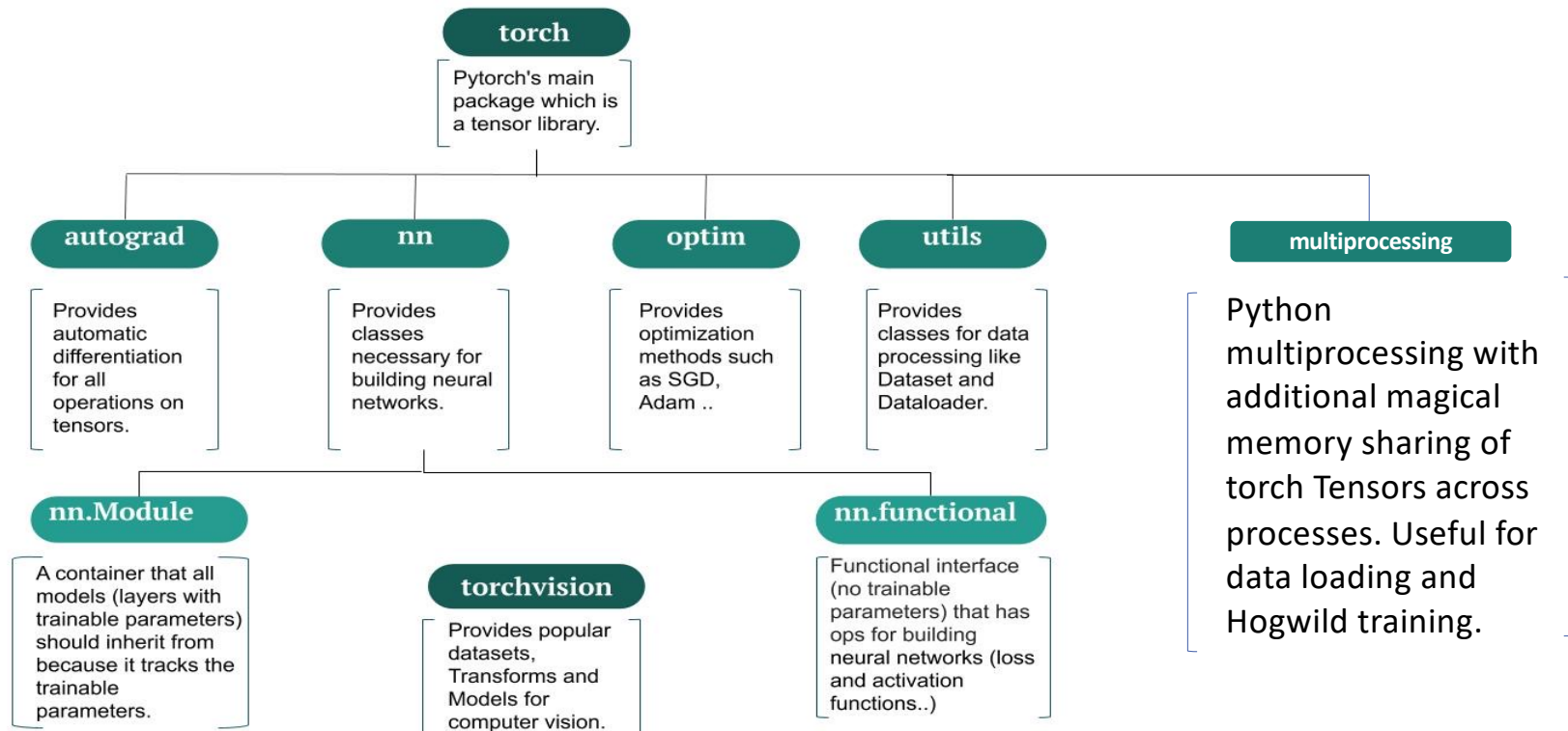
Example: Llama Hyperparameters

These hyperparameters directly impact memory footprint, communication cost, and KV-cache size during inference.



PyTorch Multiprocessing

PyTorch Key Components



Shared Memory

- **Shared memory** is a memory area that the OS (eg Linux) maps on the address space of the processes, allowing in this way to be simultaneously accessed by multiple programs with an intent to provide communication among them or avoid redundant copies.
 - It is an efficient means of passing data between programs
- Do not confuse this shared memory used as communication between OS processes with the concept of the shared memory in the GPU that is a HW component to reduce GPU memory access latency
 - see also: <https://devblogs.nvidia.com/using-shared-memory-cuda-cc/>

Python Threading

```
class threading.Thread(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)
```

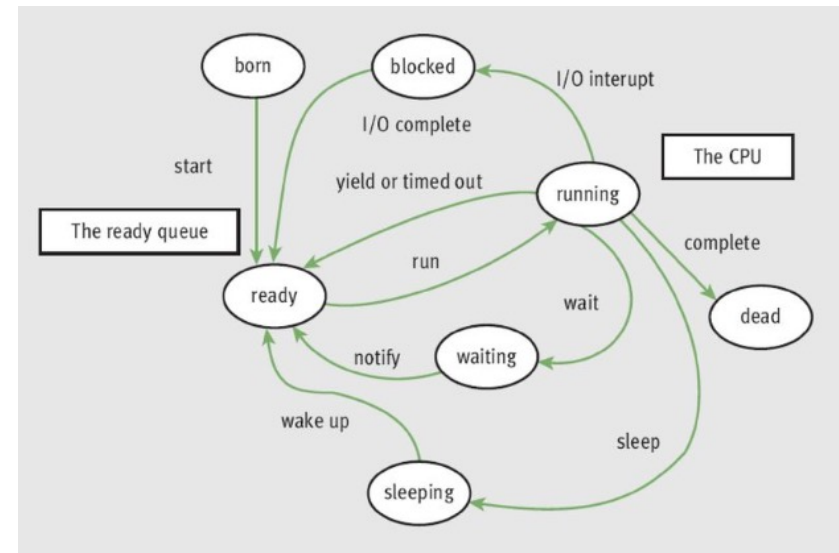
- Allows creation and handling of *Thread* objects: independent execution context
- Threads share data unless its thread local:
 - *mydata = threading.local()*
- Threads in Python:
 - **CONCURRENCY** but **NOT PARALLELISM** (global interpreter lock--GIL)
 - Within a single process only one thread may be processing Python byte-code at any one time.
 - Memory is shared unless is specifically thread local
 - Cannot take advantage of multiprocessing
- Daemon threads:
 - They do not die with the parent (wait for interpreter shutdown) or needs to be explicitly killed

Not useful for Parallelism Performance!

See <https://docs.python.org/3.6/library/threading.html#module-threading>

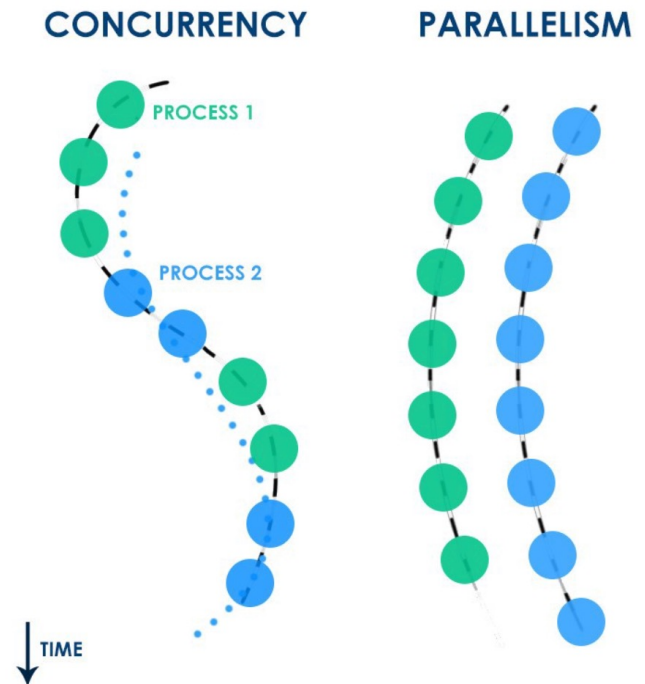
More on Python Threads – Concurrency

- In Python, a thread is
 - an object like any other in that it can hold data,
 - be run with methods,
 - be stored in data structures, and
 - be passed as parameters to methods
 - A thread can also be executed as a process
 - Before it can execute, a thread's class must implement a run method
 - During its lifetime, a thread can be in various states
-

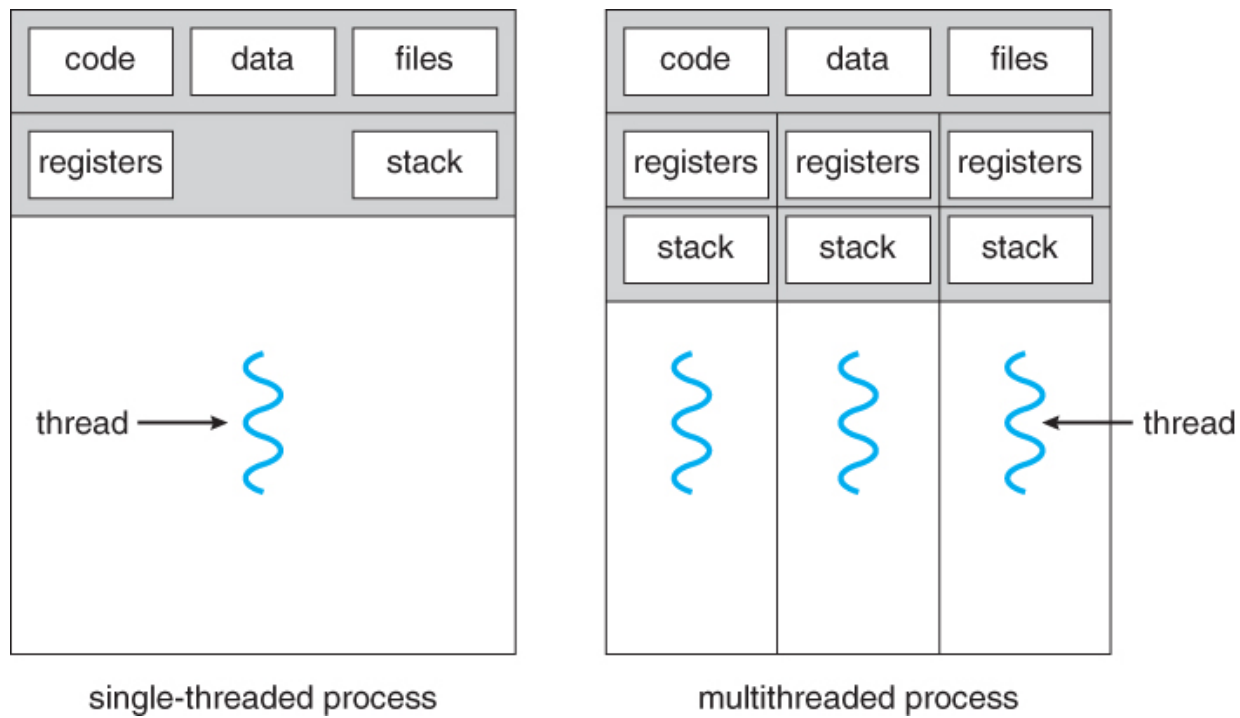


Concurrency vs. Parallelism

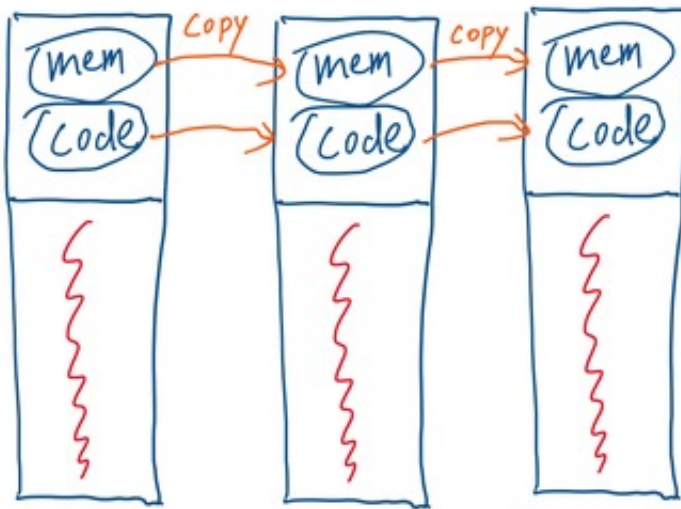
S.NO	Concurrency	Parallelism
1.	Concurrency is the task of running and managing the multiple computations at the same time.	While parallelism is the task of running multiple computations simultaneously.
2.	Concurrency is achieved through the interleaving operation of processes on the central processing unit(CPU) or in other words by the context switching.	While it is achieved by through multiple central processing units(CPUs).
3.	Concurrency can be done by using a single processing unit.	While this can't be done by using a single processing unit. it needs multiple processing units.
4.	Concurrency increases the amount of work finished at a time.	While it improves the throughput and computational speed of the system.
5.	Concurrency deals lot of things simultaneously.	While it do lot of things simultaneously.
6.	Concurrency is the non-deterministic control flow approach.	While it is deterministic control flow approach.
7.	In concurrency debugging is very hard.	While in this debugging is also hard but simple than concurrency.



Threads vs. Processes

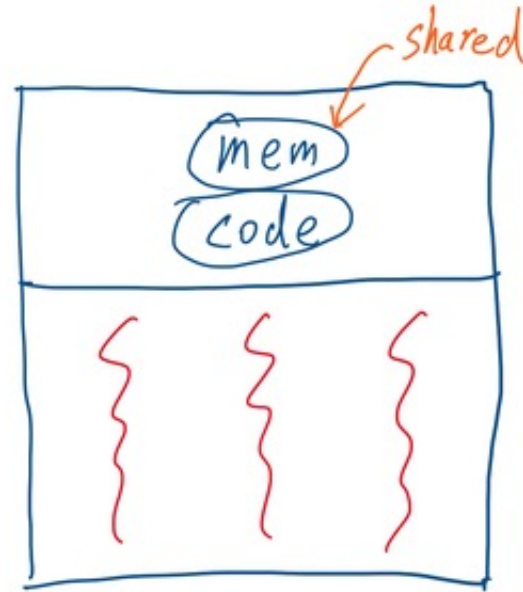


Multi-threading vs. Multi-processing



3 processes

multi-processing



1 process, 3 threads

multi-threading

Multiprocessing vs. Multithreading in Python

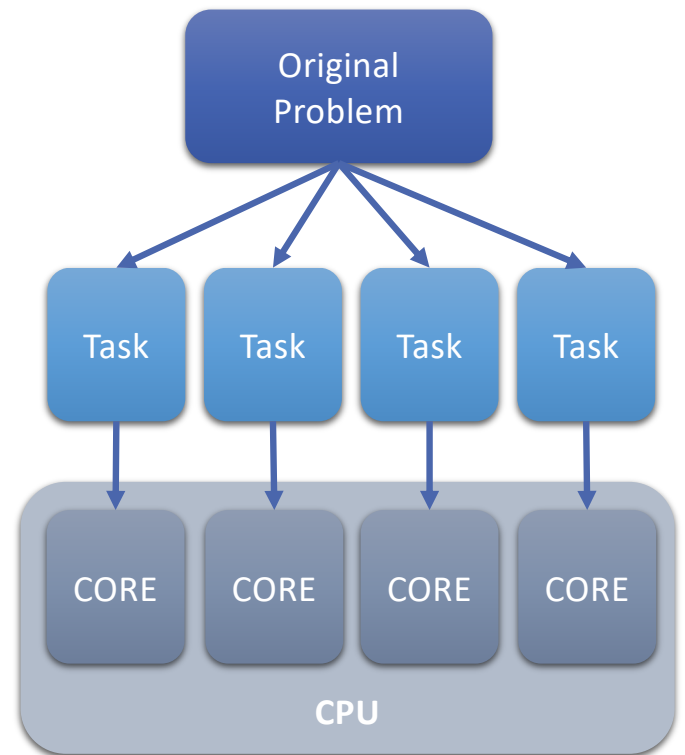
Feature	Python Multiprocessing	Python Multithreading
Execution Model	Runs separate processes	Runs multiple threads within a single process
GIL Effect	Bypasses GIL	Blocked by GIL
Memory Sharing	Independent memory space	Shared memory within process
Best for	CPU-bound tasks (e.g., ML training, video processing)	I/O-bound tasks (e.g., web scraping, network requests)
Concurrency or Parallelism?	True parallelism	Concurrency but no true parallelism
Communication	Queue, Pipe, SharedMemory	Shared memory via global variables
Crash Isolation	If one process crashes, others continue	If one thread crashes, the whole process may crash

Python GIL (Global Interpreter Lock) and Its Impact

- Python's GIL **prevents multiple threads from executing Python bytecode in parallel.**
- This means Python **threads are not useful for CPU-bound tasks** since only one thread runs at a time.
- However, threads **are useful for I/O-bound tasks** where waiting for external input (like a database or network request) dominates execution time.

Why Multiprocessing

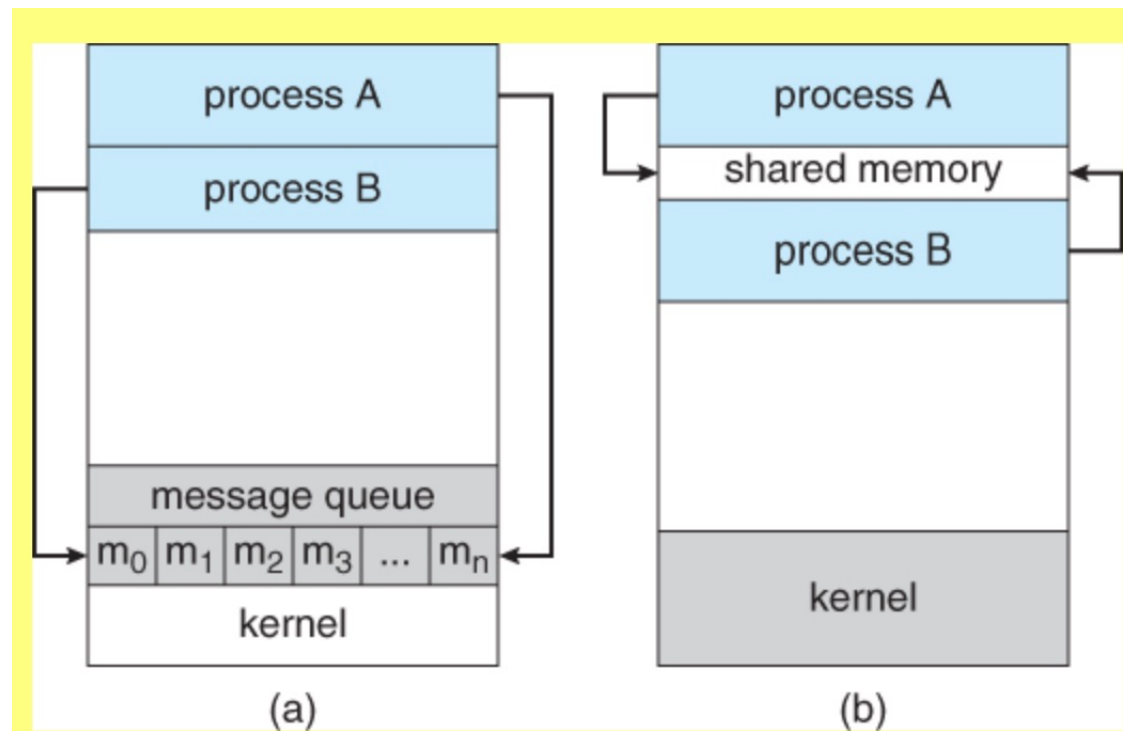
- **CPU** has many cores and hardware threads
 - Superscalar CPU cores with hardware multi-threading
- **When a problem can be parallelized (i.e. divided in parallel tasks) use parallel tasks to complete it in less time**
- Examples of parallelizable ML tasks:
 - Load multiple files from disk
 - Applying transformations to each dataset sample
 - Send/Receive data from Multiple GPUs
 - Serving concurrent inference requests
 - Data preprocessing for RAG pipelines



Inter Process Communication

Message Passing

Shared Memory



Python Multiprocessing

Overview

- Python's built-in multiprocessing module allows concurrent execution of processes, bypassing the Global Interpreter Lock (GIL) by using separate memory spaces for each process.

Key Features:

- **Process-based parallelism:** Uses multiple processes instead of threads to enable true parallel execution.
- **Inter-process communication (IPC):** Supports mechanisms like Queue, Pipe, SharedMemory, and Value/Array.

Multiple start methods:

- "spawn": Creates a new Python process and imports the module.
- "fork" (Unix only): Child process inherits memory of the parent (faster but has risks with shared states).
- "forkserver": Spawns a server process and forks new processes from it.

PyTorch Multiprocessing (torch.multiprocessing)

- **Overview**

- PyTorch extends Python's multiprocessing by integrating GPU-aware functionality and specialized mechanisms for deep learning.

- **How is it Unique?**

1. **Supports Tensor Sharing Across Processes:**

- PyTorch processes can share torch.Tensor objects in shared memory, reducing memory duplication and speeding up communication.
- Tensor.share_memory_() makes tensors accessible across processes.

2. **Optimized for GPU and Deep Learning Workloads:**

- Unlike Python multiprocessing, PyTorch's multiprocessing is designed to work efficiently with CUDA tensors.
- Ensures that operations on CUDA tensors do not create race conditions.
- Uses torch.cuda.Stream for proper synchronization when dealing with multiple GPUs.

3. **Default Start Method is 'spawn':**

- PyTorch **defaults to "spawn"** instead of "fork" to prevent issues with CUDA memory management (forking with GPU memory can lead to corruption).

4. **Distributed Processing Support (torch.distributed):**

- PyTorch integrates with torch.distributed to allow multiple processes to communicate efficiently, enabling distributed training.

Example (CPU-based)

```
1 import torch
2 import torch.multiprocessing as mp
3
4 def worker(tensor):
5     tensor += 1 # Modify shared tensor
6     print(f"Updated Tensor: {tensor}")
7
8 if __name__ == "__main__":
9     mp.set_start_method("spawn") # Ensure compatibility
10    shared_tensor = torch.zeros(3).share_memory_()
11
12    processes = []
13    for _ in range(3):
14        p = mp.Process(target=worker, args=(shared_tensor,))
15        p.start()
16        processes.append(p)
17
18    for p in processes:
19        p.join()
20
21    print(f"Final Tensor: {shared_tensor}")
22
```

Sample Output

```
Updated Tensor: tensor([1., 1., 1.])
Updated Tensor: tensor([2., 2., 2.])
Updated Tensor: tensor([3., 3., 3.])
Final Tensor: tensor([3., 3., 3.])
```

PyTorch Multiprocessing in Deep Learning

PyTorch's multiprocessing is particularly useful for:

- **Data Loading (`torch.utils.data.DataLoader`):** Uses `num_workers > 0` to parallelize data loading.
- **Model Parallelism:** Splits model computation across multiple processes.
- **Distributed Training (`torch.distributed.launch`):** Manages multi-GPU and multi-node training.

Python Multiprocessing and Pool examples

- Example: creation of a *process*
- Example: creation of a *pool*

```
from multiprocessing import Process, Pool

def f(name):
    print('hello', name)

def f(x):
    return x*x

if __name__ == '__main__':
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()

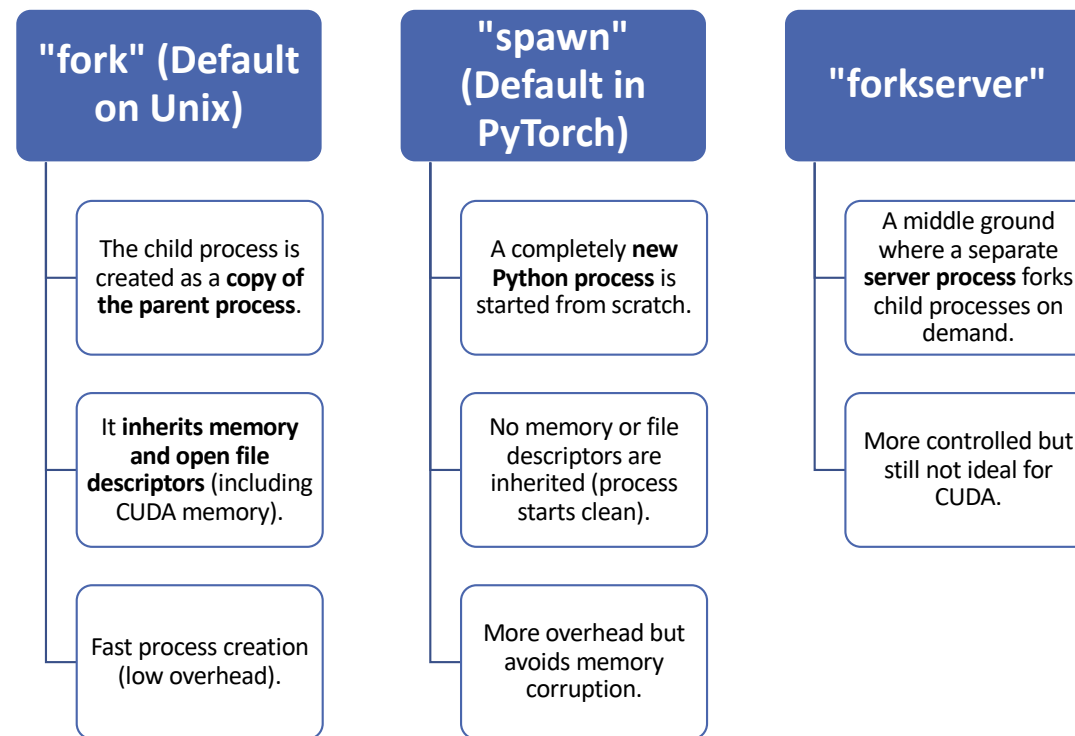
    with Pool(5) as p:
        print(p.map(f, [1, 2, 3]))
```


Python vs. PyTorch Multiprocessing

Feature	Python multiprocessing	PyTorch multiprocessing
Main Purpose	General parallel processing	Deep learning and GPU-aware processing
Memory Sharing	Processes have independent memory	Supports shared tensors (efficient for ML)
GPU Support	No direct support for CUDA	Designed for CUDA operations
Default Start Method	"fork" (Unix), "spawn" (Windows)	"spawn" (prevents CUDA issues)
Communication	Queue, Pipe, SharedMemory	Tensor-aware shared memory
Distributed Training	Not natively supported	torch.distributed for multi-node training
Use Case	CPU-based parallel tasks	Model training, inference, and data loading

Understanding Fork vs. Spawn in Python Multiprocessing

Python provides three ways to create new processes:



How Fork Works and Why It Fails with CUDA

With "fork", the child process inherits all the **parent's memory state**, including:

- **CUDA runtime state**
- **GPU allocations**
- **Open CUDA streams**
- **CuDNN/cublas handles**

Since CUDA **does not support process forking**, this causes **memory corruption** in the child process.

Why Forking CUDA Memory is Dangerous

- When a child process inherits CUDA memory:
 - **The child process does not own the memory.** It tries to use the parent's CUDA buffers.
 - **CUDA memory manager is not designed for forked processes.** CUDA maintains a **single global state** that cannot be shared safely across processes.
 - **Lazy initialization of CUDA in PyTorch** can cause inconsistencies when a forked process attempts to use CUDA.
- **CUDA Runtime Restriction:**
 - **"The CUDA context is not fork-safe, and using CUDA in a forked process can lead to undefined behavior."** — NVIDIA Documentation

Why PyTorch Uses "spawn"

- PyTorch **forces "spawn"** as the default to ensure **safe CUDA memory management**.
- With "spawn":
 - Each process **starts fresh**, meaning:
 - No inherited CUDA memory.
 - No risk of GPU memory corruption.
 - CUDA is **re-initialized properly** for each process.
 - PyTorch safely re-allocates GPU tensors **per process** instead of sharing memory.

```
1
2 import torch
3 import torch.multiprocessing as mp
4
5 def worker():
6     tensor = torch.randn(3).cuda() # Allocate safely inside the child process
7     print(f"Worker sees tensor: {tensor}")
8
9 if __name__ == "__main__":
10     mp.set_start_method("spawn") # Safe for CUDA
11     p = mp.Process(target=worker)
12     p.start()
13     p.join()
14
```

Summary

Start Method	Behavior	CUDA Safe?	Process Creation Speed
fork	Inherits parent memory (fast)	 Unsafe for CUDA (corruption risk)	 Fast
spawn	Starts a new clean process	 Safe for CUDA	 Slower
Forkserver	Uses a separate process to fork children	 Still unsafe for CUDA	 Moderate

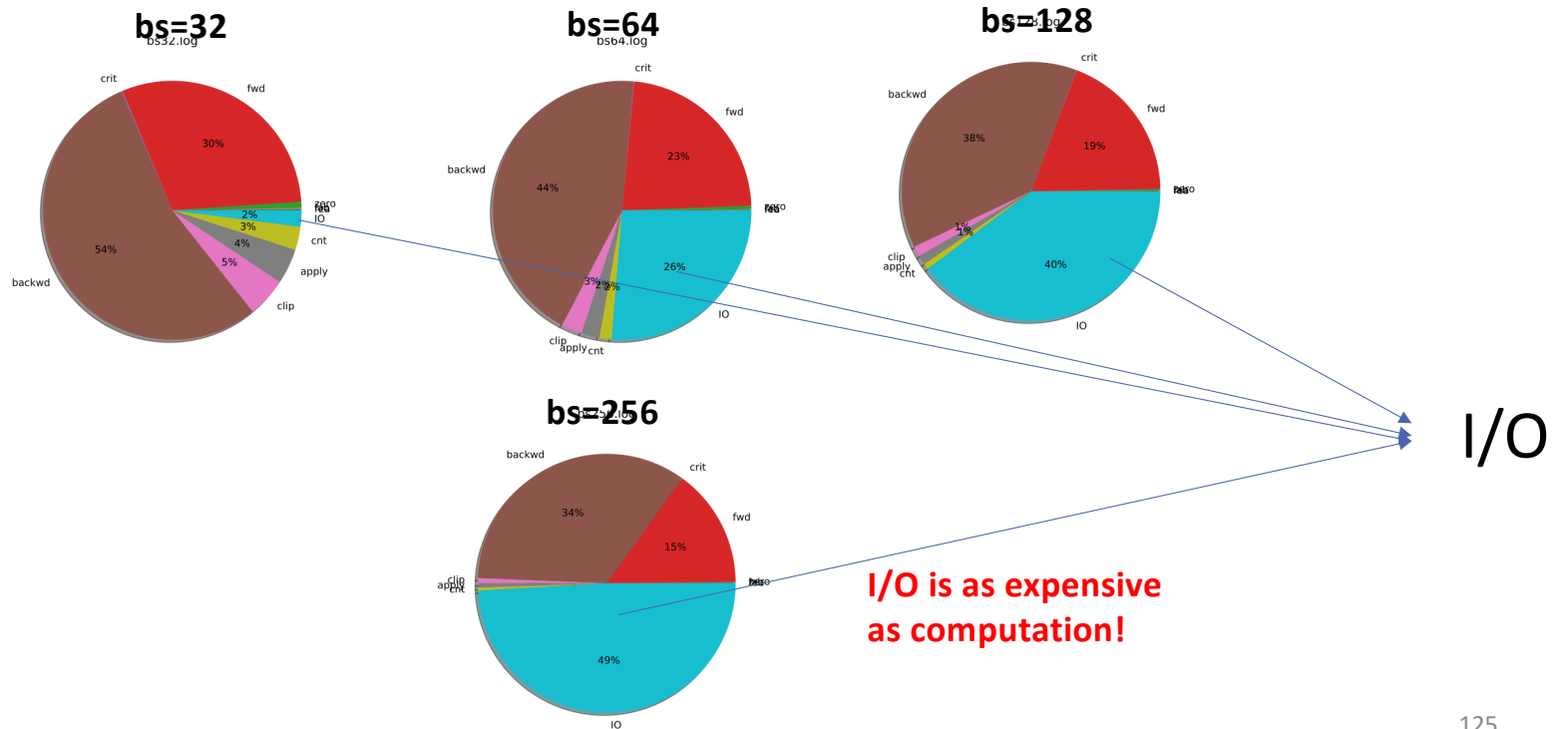
Comparison of Multiprocessing Communication Mechanisms

Mechanism	Type	Best For	Pros	Cons
Queue	FIFO Message Passing	Many-to-Many Process Communication	Simple, process-safe	High overhead (pickling/unpickling)
Pipe	Direct Communication	Two-Process Communication	Low overhead	Limited to 2 processes
SharedMemory	Direct Memory Access	Fast Shared Data (Large Tensors, NumPy)	No copying overhead	Manual cleanup required
Value/Array	Shared Primitive Variables	Small Counters or Flags	Low memory use	Limited to simple data types

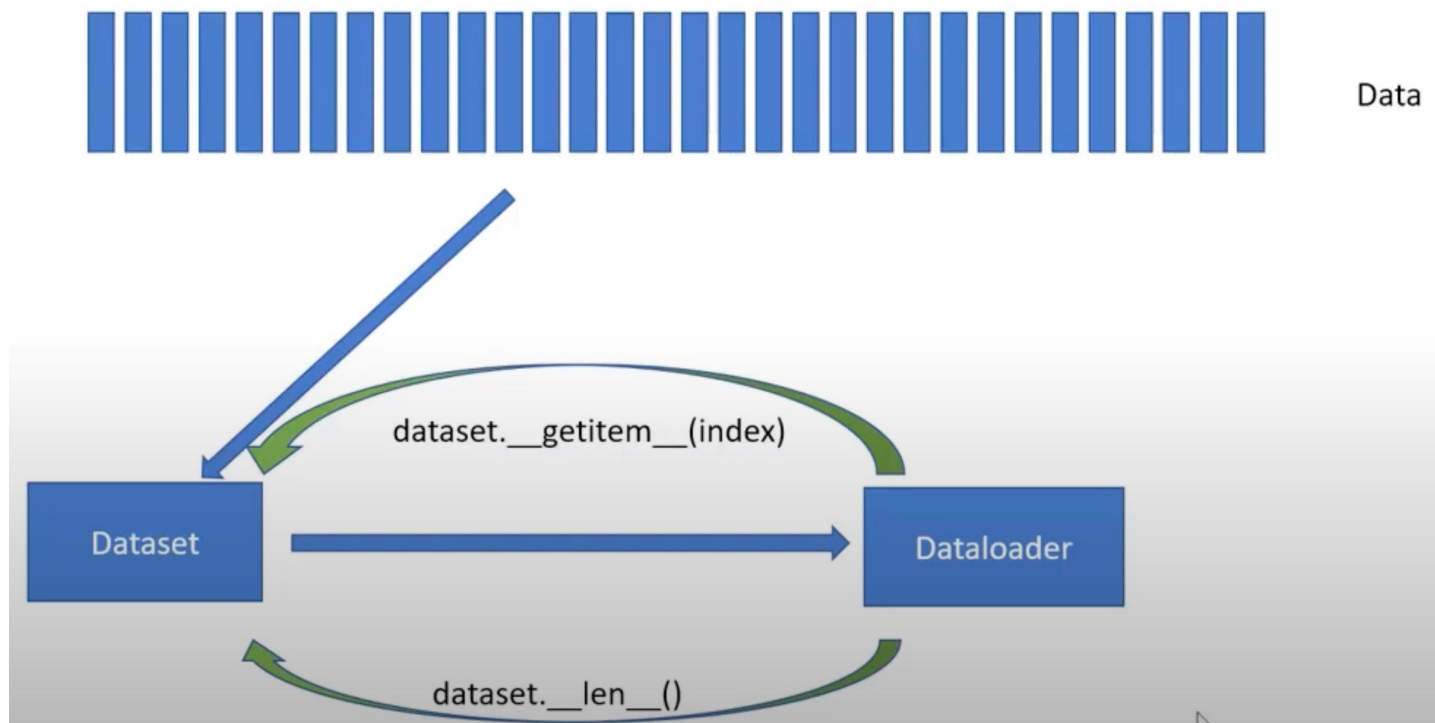
PyTorch Data loading and Preparation

I/O could be a big problem

- As GPU gets faster, I/O becomes a bottleneck



Datasets and Dataloaders



Loading data - *torch.utils.data.Dataset*

- *torch.utils.data.Dataset* represents a dataset (abstract class)
- Create your own class and override the following:
 - `__len__` so that `len(dataset)` returns the size of the dataset.
 - `__getitem__` to support the indexing such that `dataset[i]` can be used to get i-th sample
- `__getitem__` will be called to load 1 item at a time. Returns a fixed-length token sequence.
 - Data is read using memory-mapped files (mmap).
 - Tokenization is done offline to remove runtime overhead.
- For LLMs, data loading is bandwidth-bound, not compute-bound.

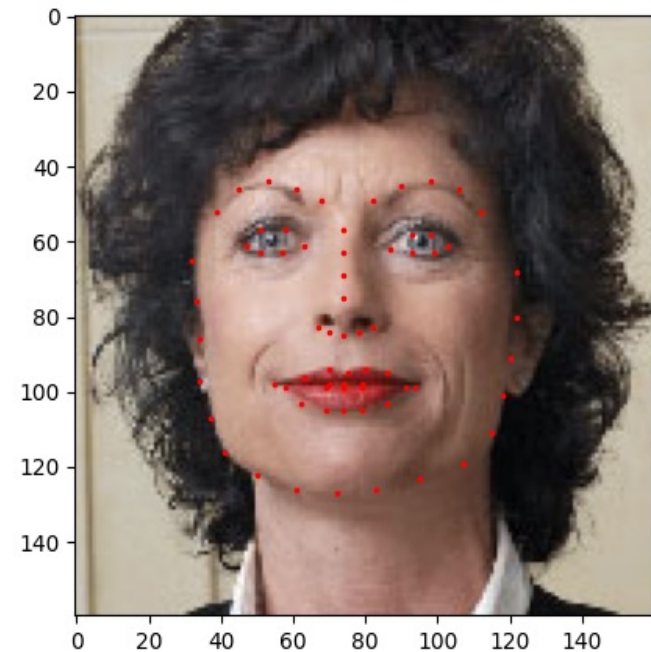
Landmark dataset

Dataset comes with a csv file with annotations which looks this:

image_name,part_0_x,part_0_y,part_1_x,part_1_y,part_2
... ,part_67_x,part_67_y 0805

personali01.jpg,27,83,27,98, ... 84,134

1084239450_e76e00b7e7.jpg,70,236,71,257, ... ,128,312



68 different landmark points are annotated for each face.

Custom Dataset Class Example

- `torch.utils.data.Dataset` is an abstract class representing a dataset. Your custom dataset should inherit `Dataset` and override the following methods:
- `__len__` so that `len(dataset)` returns the size of the dataset.
- `__getitem__` to support the indexing such that `dataset[i]` can be used to get *i*th sample.
- Define a set of transformation primitives for each sample
- Pass the transformation primitives through the *transform* argument

```
class FaceLandmarksDataset(Dataset):
    """Face Landmarks dataset."""

    def __init__(self, csv_file, root_dir, transform=None):
        """
        Args:
            csv_file (string):
                Path to the csv file with annotations.
            root_dir (string):
                Directory with all the images.
            transform (callable, optional):
                Optional transform to be applied on a sample.
        """
        self.landmarks_frame = pd.read_csv(csv_file)
        self.root_dir = root_dir
        self.transform = transform
```

from: http://pytorch.org/tutorials/beginner/data_loading_tutorial.html#dataset-class

Custom Dataset Class Example

- Transform primitives are applied in `__getitem__` to each sample after loading it with `io.imread()`

```
def __len__(self):
    return len(self.landmarks_frame)

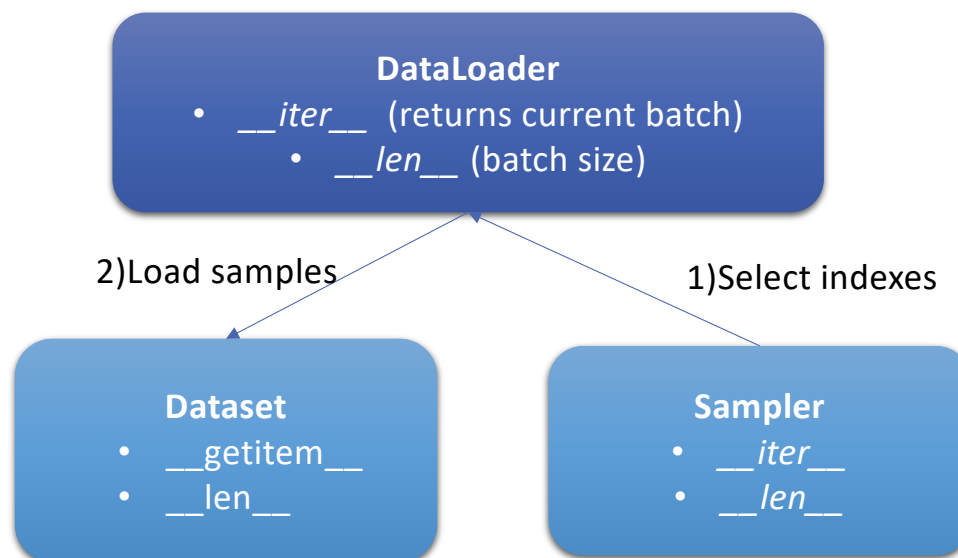
def __getitem__(self, idx):
    img_name = os.path.join(self.root_dir,
                             self.landmarks_frame.iloc[idx, 0])
    image = io.imread(img_name)
    landmarks = self.landmarks_frame.iloc[idx, 1:]
    landmarks = np.array([landmarks])
    landmarks = landmarks.astype('float').reshape(-1, 2)
    sample = {'image': image, 'landmarks': landmarks}
    if self.transform:
        sample = self.transform(sample)

    return sample
```

from: http://pytorch.org/tutorials/beginner/data_loading_tutorial.html#dataset-class

Loading data - *torch.utils.data.DataLoader*

- Selects and Loads data from the Dataset
- Composed of a **Dataset** and a **Sampler**



PyTorch: DataLoaders

A **DataLoader** wraps a **Dataset** and provides mini-batching, shuffling, multithreading, for you

When you need to load custom data, just write your own Dataset class

```
import torch
from torch.utils import data

class Dataset(data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDs, labels):
        'Initialization'
        self.labels = labels
        self.list_IDs = list_IDs

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDs)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDs[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```


PyTorch: DataLoaders

```
# Parameters
params = {'batch_size': 64,
          'shuffle': True,
          'num_workers': 6}
max_epochs = 100

# Datasets
partition = # IDs
labels = # Labels

# Generators
training_set = Dataset(partition['train'], labels)
training_generator = data.DataLoader(training_set, **params)

validation_set = Dataset(partition['validation'], labels)
validation_generator = data.DataLoader(validation_set, **params)

# Loop over epochs
for epoch in range(max_epochs):
    # Training
    for local_batch, local_labels in training_generator:
        # Transfer to GPU
        local_batch, local_labels = local_batch.to(device), local_labels.to(device)

        # Model computations
        [...]
```

PyTorch DataLoader Capabilities

1. **Define a dataset to work with:** identifying where the data is coming from and how it should be accessed.
2. **Batch the data:** define how many training or testing samples to use in a single iteration. Because data are often split across training and testing sets of large sizes, being able to work with batches of data can allow your training and testing processes to be more manageable.
3. **Shuffle the data:** PyTorch can handle shuffling data for you as it loads data into batches. This can increase representativeness in your dataset and prevent accidental skewness.
4. **Support multi-processing:** PyTorch is optimized to run multiple processes at once to make better use of modern CPUs and GPUs and to save time in training and testing your data. The DataLoader class lets you define how many workers should go at once.
5. **Merge datasets:** optionally, PyTorch also allows you to merge multiple datasets together. While this may not be a common task, having it available is a great feature.
6. **Load data directly on CUDA tensors:** because PyTorch can run on the GPU, you can load the data directly onto the CUDA before they're returned.

Loading data - *torch.utils.data.Dataloader*

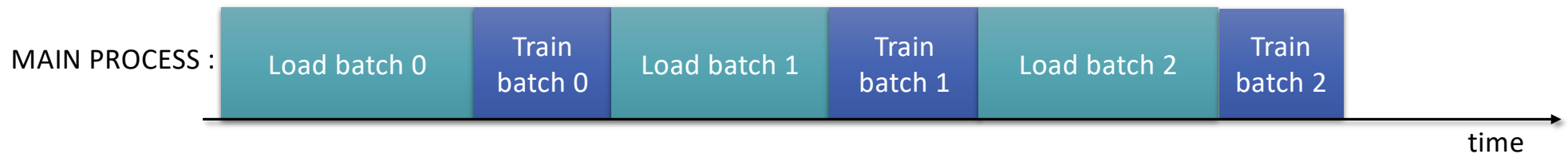
- Useful Parameters:
 - *batch_size*: batch size
 - *sampler*: define which sampler to use
 - *pin_memory*: copy into CUDA pinned memory before returning the data
 - *shuffle*: reshuffle data at each epoch
- Available *samplers*:
 - *SequentialSampler*
 - *RandomSampler*
 - *SubsetRandomSampler*
 - *WeighetRandomSampler*
 - Create your *CustomSampler*

Pinned Memory

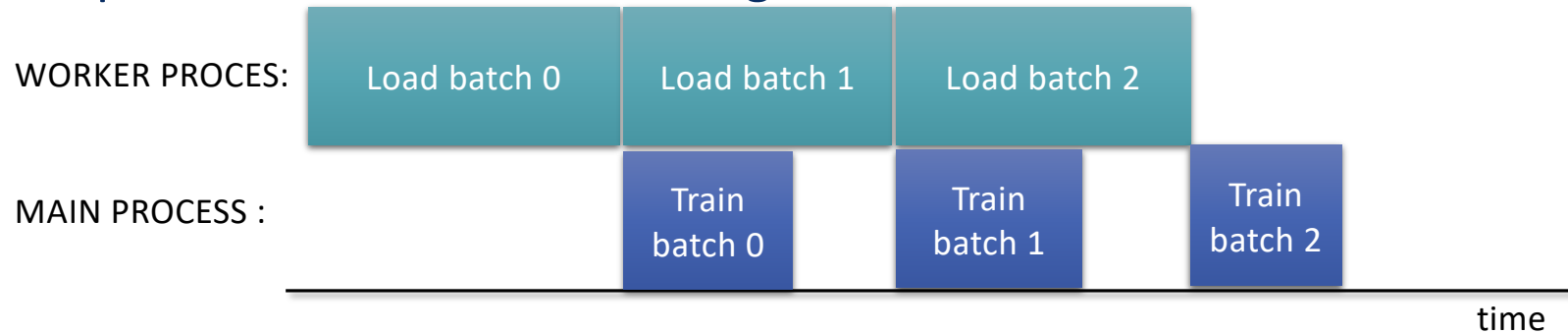
- **Pinned memory** (also known as page-locked memory) is located in the **CPU (host)** memory, not in the GPU memory. However, it has a special role in facilitating faster **data transfers between the CPU and GPU**.
- It is **page-locked** (cannot be swapped out to disk by the OS).
- Allows **faster data transfers** between CPU and GPU via **direct memory access (DMA)**.
- Reduces latency by avoiding intermediate copies during transfers.
- Commonly used to **optimize GPU-CPU data transfer** in deep learning and high-performance computing.

Data Prefetching

- Normal pipeline:



- Pipeline with Load Prefetching:

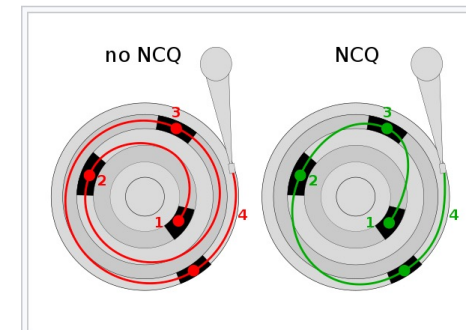


Load time still limiting factor. Can we do better?

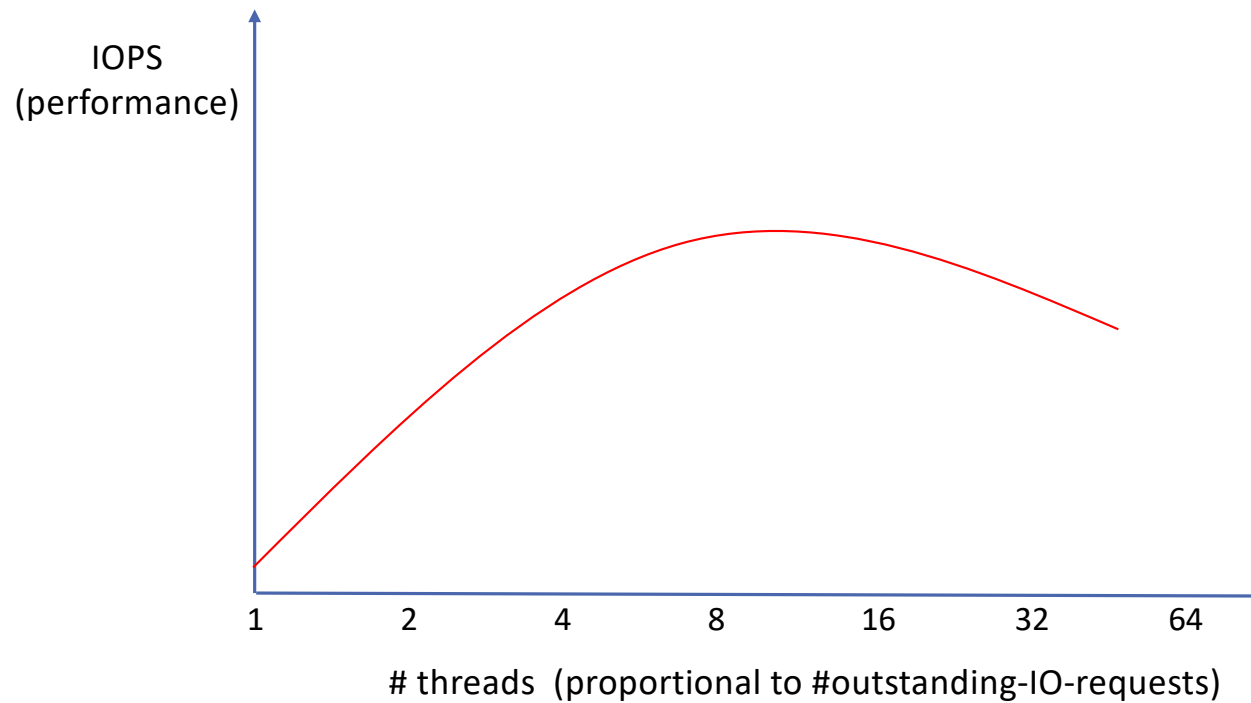
Disk Performance Characteristics

- IOPS: IO-ops/sec
 - read/write
 - random/sequential
- Throughput = IOPS * BlockSize [bytes/sec]
 - Block size: 4KB+
- Queue Depth: maximum number of Outstanding IO requests in a device
- **Important fact:** IOPS can be higher with multiple I/O outstanding requests
 - Tagged Command Queueing (TCQ) and Native Command Queueing (NCQ) techniques

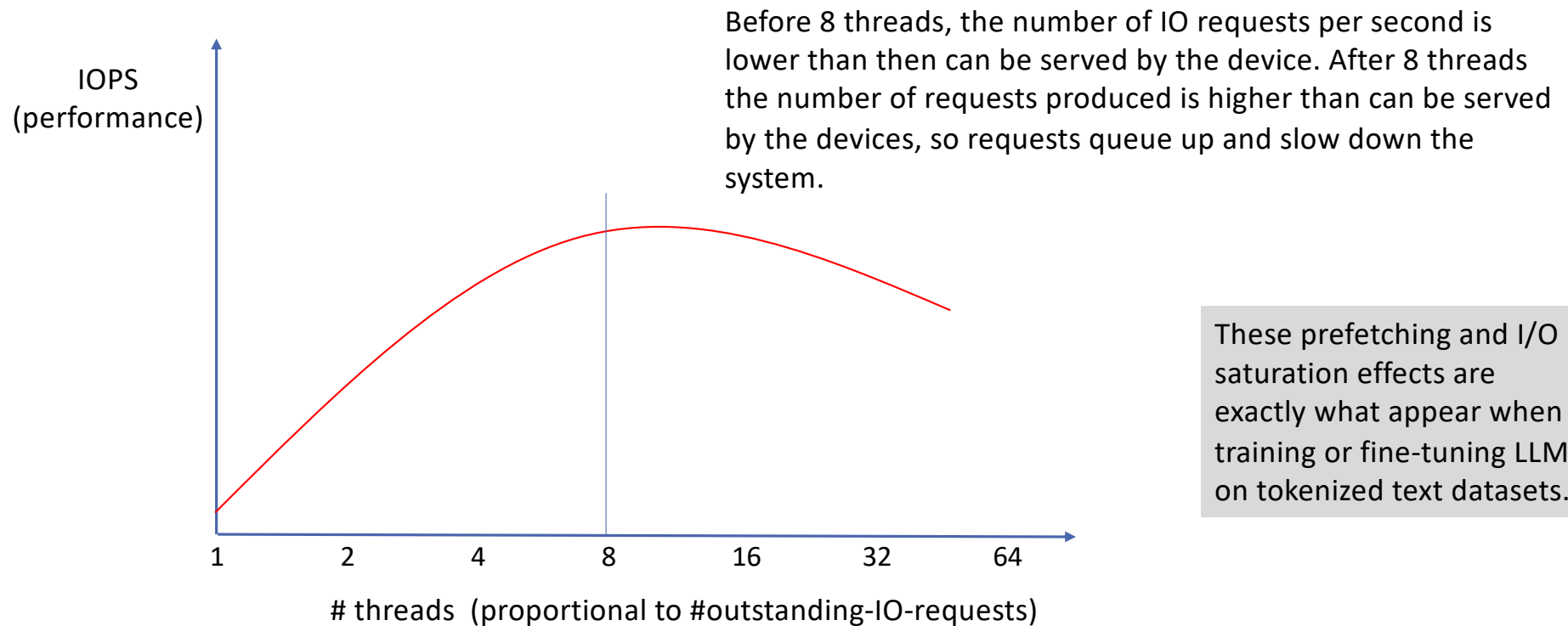
In a file system, a *block* is the largest contiguous amount of disk space that can be allocated to a file and the largest amount of data that can be transferred in a single I/O operation. The block size determines the maximum size of a read request or write request that a file system sends to the I/O device driver.



Disk Performance Characteristics



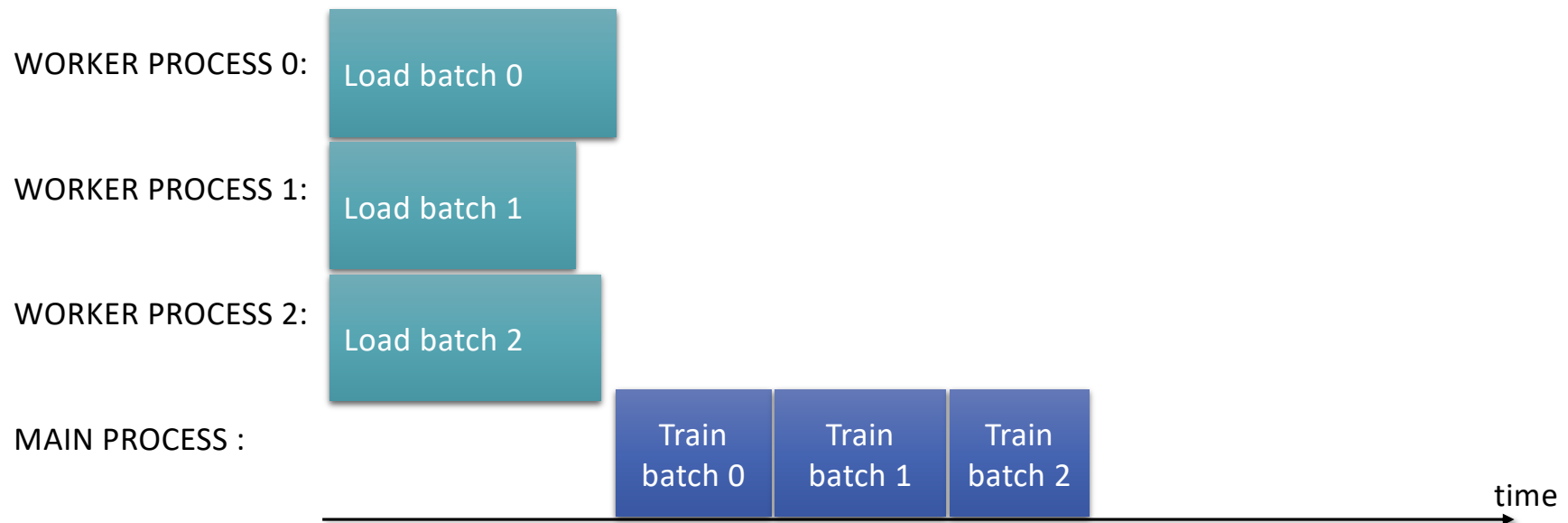
Disk Performance Characteristics



These prefetching and I/O saturation effects are exactly what appear when training or fine-tuning LLMs on tokenized text datasets.

Multi-threaded Data Prefetching

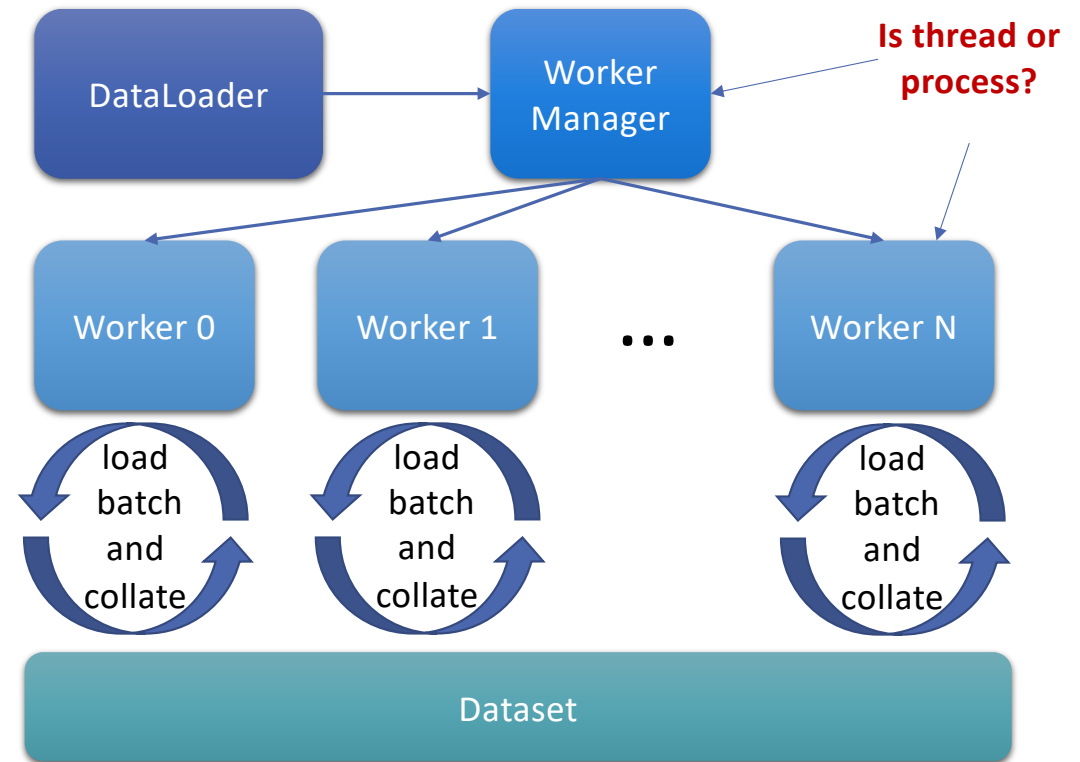
- Issue many outstanding I/O requests using multiple threads:



- Prefetching limitations: buffer memory, outstanding I/O requests limit, predicting next needed address

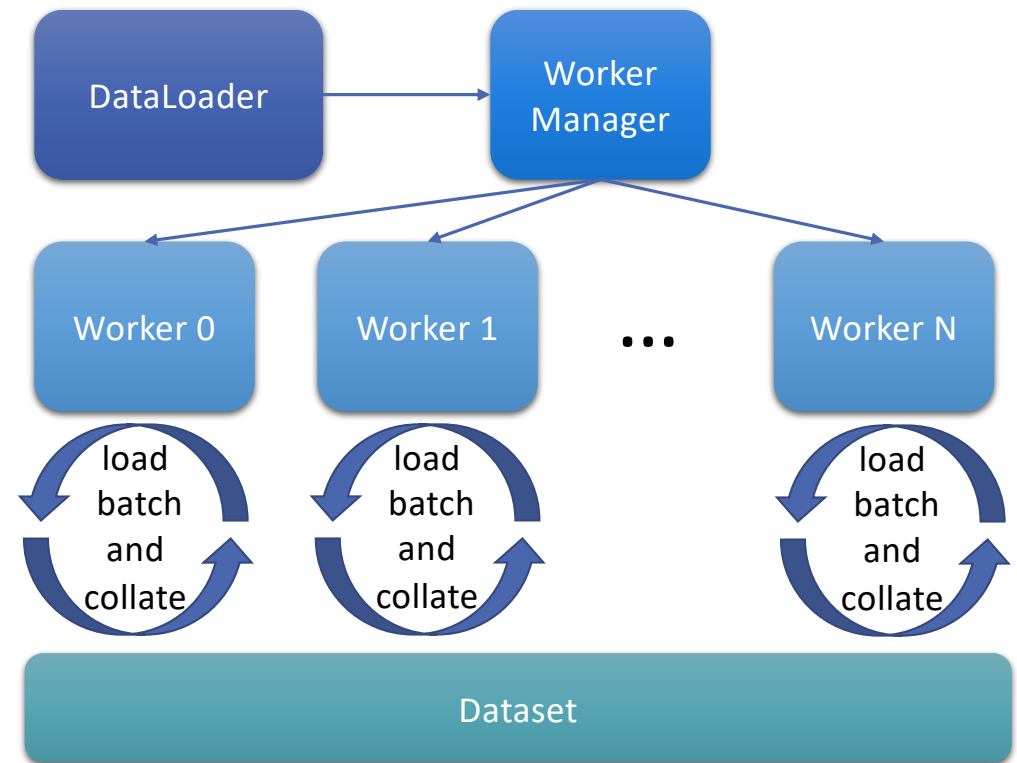
torch.utils.data.Dataloader Architecture

- DataLoader (main process):
 - Requests a set of batches based on *sampler*
- WorkerManager:
 - Dispatch batches to workers
- Workers :
 - Load all samples in each batch
 - Collate batches:
 - Build a batch Tensor with samples
- Synchronization:
 - Producer/Consumer using shared queues and Signals/Exceptions



torch.utils.data.Dataloader Architecture

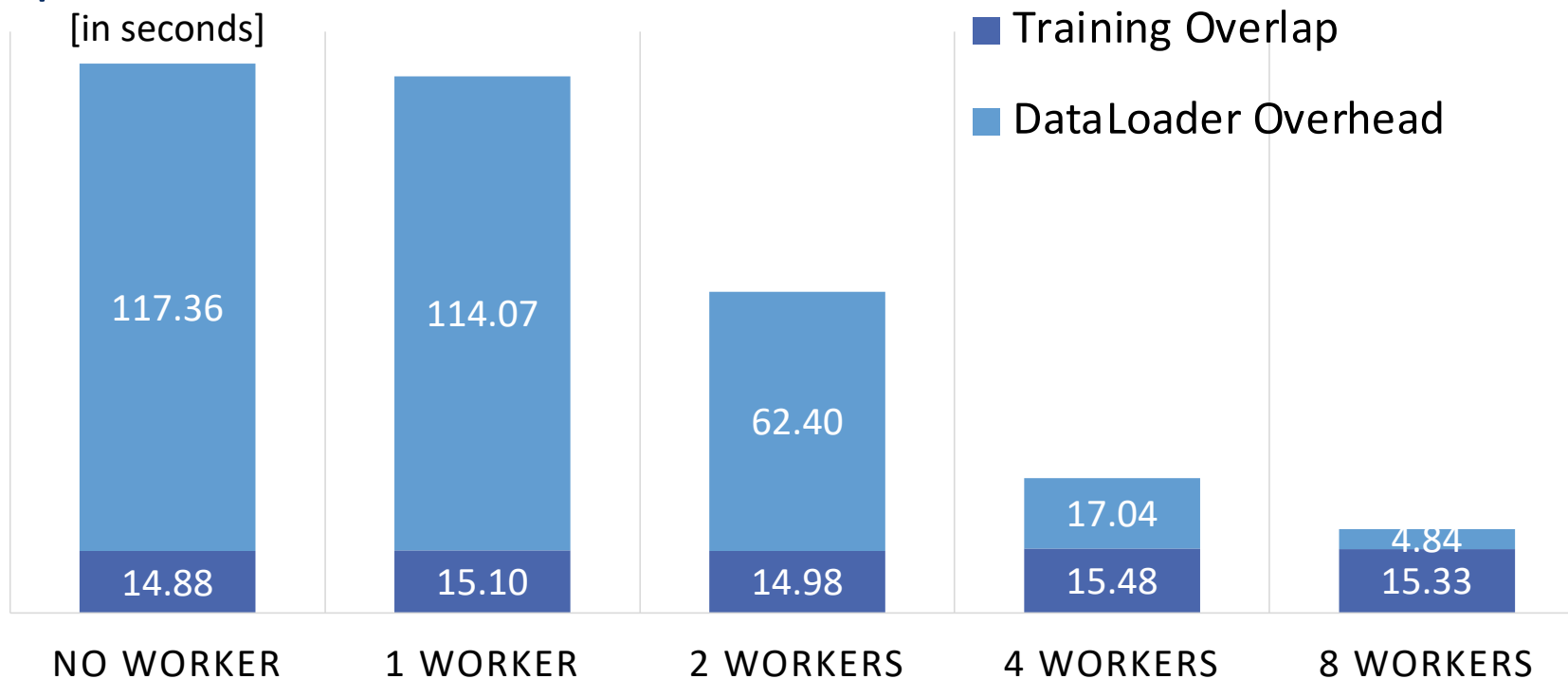
- DataLoader (main process):
 - Requests a set of batches based on *sampler*
- WorkerManager (**thread**):
 - Dispatch batches to workers
- Workers (**processes**):
 - Load all samples in each batch
 - Collate batches:
 - Build a batch Tensor with samples
- Synchronization:
 - Producer/Consumer using shared queues and Signals/Exceptions



Example: Token Shard Loading for LLM Training

- Dataset: tokenized web-scale text
- Storage: NVMe SSD
- batch_size = 2048 tokens, num_workers = 0 vs 8
- Most time spent in:
 - Disk I/O
 - Memory copies
- Key question:
 - How many DataLoader workers saturate NVMe bandwidth?

Example: Number of workers' effect



- Ideal worker scaling limited by:
 - Storage bandwidth
 - Queue depth
 - CPU memory bandwidth
- Beyond saturation, workers compete rather than help.

Lesson Key Points

- ML Performance Factors
- Performance Optimization Methodology:
 1. Measurement: Metrics, Time/Resources and Techniques
 2. Analysis: Amdahl's Law, Bandwidth/Latency, Roofline Model
 3. Optimization (in relationship to Roofline model)
- PyTorch Optimizer Algorithms
- PyTorch Multiprocessing
- PyTorch DataLoader and Disk performance
- Modern AI performance is dominated by data movement, memory behavior, and orchestration, not just compute.

Appendix

ML Performance Factors



Algorithm Performance

- **Training vs. Inference Performance:** Training focuses on throughput using backpropagation and large batch GEMMs, while inference prioritizes low-latency token generation dominated by matrix-vector operations and memory movement.
- **Distributed Training Algorithms:** Using data parallelism, model parallelism, pipeline parallelism, FSDP, or ZeRO to scale training across many accelerators.
- **Inference-Centric Algorithms:** Techniques such as speculative decoding and early-exit decoding reduce the number of expensive forward passes.
- **Attention Variants:** FlashAttention and paged attention reduce memory bandwidth and improve locality for long sequences.
- **KV-Cache Optimization:** Storing, compressing, quantizing, or evicting key-value tensors to reduce memory footprint and bandwidth.
- **Efficient Inference Algorithms:** Applying INT8/FP8/INT4 quantization, structured pruning, and sparsity to reduce compute and memory cost.

ML Performance Factors

Algorithm Performance

- Training vs. Inference Performance
- Distributed Training Algorithms. Ex: Asynchronous vs. Synchronous Distributed SGD
- Inference-Centric Algorithms (Prefill vs. Decode, Speculative Decoding)
- KV-Cache Optimization & attention variants
- Algorithms for Efficient Inference. Ex: Quantization and Pruning

Hyperparameters Performance

- Learning Rate & Schedulers, Network Depth & Width, Batch Size & Microbatch Size, Context Length / Sequence Length, KV-Cache Precision, Inference Sampling Parameters (Temperature, Top-k, Top-p)

Implementation Performance

- Algorithms Implementation (vectorization, comm./comp. overlapping, parallelization, data access)

Framework Performance

- ML Frameworks: PyTorch, TensorFlow, Caffe, MXNET.
- Graph compilers and Distributed Runtimes
- Inference serving frameworks

Library Performance

- Math libraries (cuDNN), Communication Libraries (MPI, NCCL, GLU), Transformer libraries

Hardware Performance

- CPU, DRAM, GPU, HBM, Storage, Tensor Cores/Matrix Engines, Disk/Filesystem, Network/Interconnect

Hyperparameters Performance

- **Learning Rate & Schedulers:**
Cosine decay, warmup, and adaptive schedules strongly affect convergence speed and stability.
- **Network Dimensions:**
Depth and width determine model capacity but directly increase memory and compute cost.
- **Batch Size & Microbatch Size:**
Larger batches improve hardware utilization but may hurt convergence or exceed memory limits.
- **Context Length:**
Longer sequences significantly increase attention cost and KV-cache size.
- **KV-Cache Precision:**
Choosing FP16, FP8, or INT8 trades accuracy for memory footprint and bandwidth.
- **Inference Sampling Parameters:**
Temperature, top-k, and top-p influence output quality and sometimes runtime behavior.

ML Performance Factors

Algorithm Performance

- Training vs. Inference Performance
- Distributed Training Algorithms. Ex: Asynchronous vs. Synchronous Distributed SGD
- Inference-Centric Algorithms (Prefill vs. Decode, Speculative Decoding)
- KV-Cache Optimization & attention variants
- Algorithms for Efficient Inference. Ex: Quantization and Pruning

Hyperparameters Performance

- Learning Rate & Schedulers, Network Depth & Width, Batch Size & Microbatch Size, Context Length / Sequence Length, KV-Cache Precision, Inference Sampling Parameters (Temperature, Top-k, Top-p)

Implementation Performance

- Algorithms Implementation (vectorization, comm./comp. overlapping, parallelization, data access)

Framework Performance

- ML Frameworks: PyTorch, TensorFlow, Caffe, MXNET.
- Graph compilers and Distributed Runtimes
- Inference serving frameworks

Library Performance

- Math libraries (cuDNN), Communication Libraries (MPI, NCCL, GLU), Transformer libraries

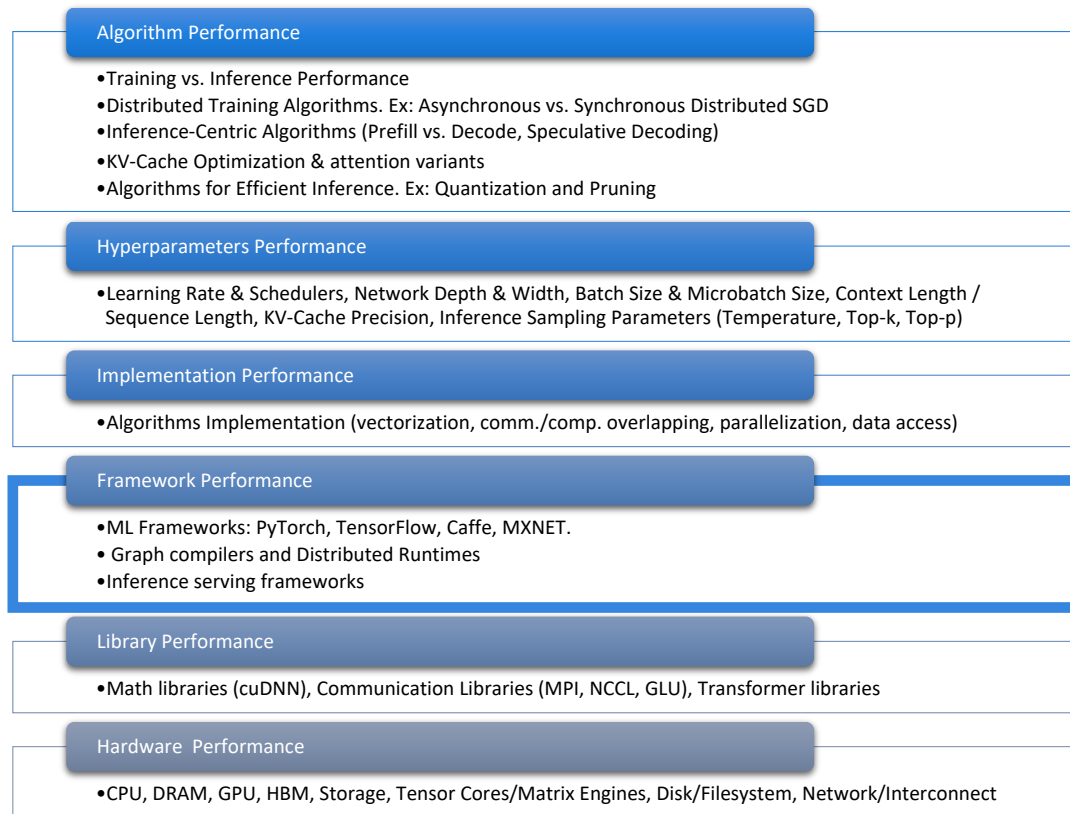
Hardware Performance

- CPU, DRAM, GPU, HBM, Storage, Tensor Cores/Matrix Engines, Disk/Filesystem, Network/Interconnect

Implementation Performance

- **Vectorization & Kernel Fusion:**
Fusing multiple operations into single kernels reduces launch overhead and memory traffic.
- **Communication / Computation Overlap:**
Hiding communication latency behind computation in distributed training.
- **Parallelization:**
Splitting data or model parameters across devices to scale performance.
- **Memory Planning & Buffer Reuse:**
Pre-allocating and reusing buffers to reduce dynamic allocations.
- **Data Access & Input Pipelines:**
Prefetching, caching, and asynchronous loading using DataLoader or tf.data.

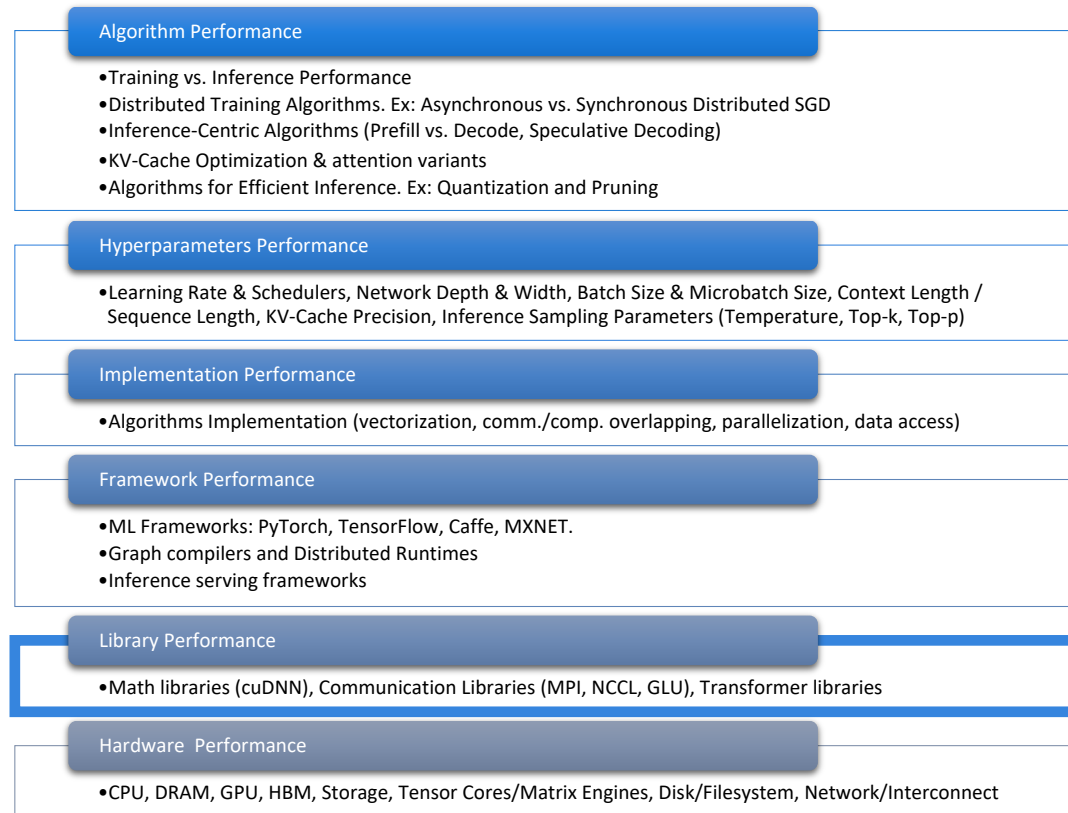
ML Performance Factors



Framework Performance

- **PyTorch:**
Using torch.compile and Inductor for graph-level and kernel-level optimizations.
- **TensorFlow / JAX:**
Using XLA for just-in-time and ahead-of-time compilation.
- **Distributed Runtime APIs:**
FSDP and ZeRO shard parameters and optimizer states across devices.
- **Model Export & AOT Compilation:**
Exporting models for ahead-of-time optimization and deployment.
- **Inference Serving Frameworks:**
vLLM, TGI, and TensorRT-LLM provide optimized batching, KV-cache handling, and scheduling.

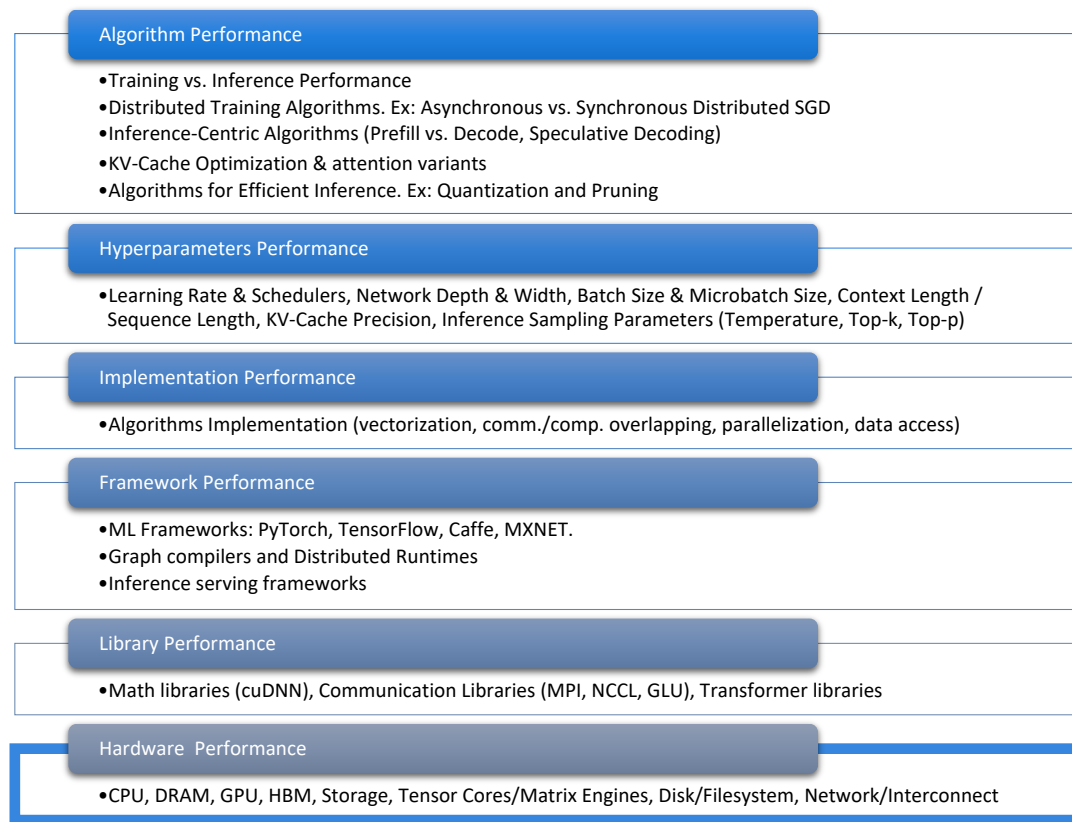
ML Performance Factors



Library Performance

- **Math Libraries:**
cuDNN, cuBLASLt, CUTLASS, oneDNN accelerate convolutions and GEMMs.
- **Transformer-Specific Libraries:**
FlashAttention, FasterTransformer, TensorRT-LLM optimize attention and decoding.
- **Communication Libraries:**
NCCL, MPI, RCCL provide fast collective communication.

ML Performance Factors



Hardware Performance

- **CPU:** Handles data preprocessing, orchestration, and input pipelines.
- **DRAM:** System memory capacity and bandwidth affect data feeding speed.
- **GPU / Accelerator:** Provides massively parallel compute for matrix operations.
- **HBM / On-package Memory:** High-bandwidth memory is critical for feeding tensor cores.
- **Tensor Cores / Matrix Engines:** Specialized units for mixed-precision matrix math.
- **Storage:** NVMe SSDs reduce dataset loading latency.
- **Network / Interconnect:** NVLink, PCIe Gen5, and RDMA enable fast multi-node scaling.

Reference: Practical Use Cases of Means in System Performance

Arithmetic Mean in System Performance:

- **Latency:** When measuring the time it takes for a system to complete a task, the arithmetic mean is often used. For example, measuring the average time it takes for a network packet to travel from one endpoint to another.
- **Execution Time:** If a CPU or GPU runs a specific task multiple times, the arithmetic mean of execution times provides a direct indication of how fast the task completes on average.

Harmonic Mean in System Performance:

- **Throughput:** When calculating the throughput of different network paths or parallel links, the harmonic mean accounts for the fact that the slowest link (or bottleneck) will dominate the performance. This provides a more accurate reflection of the system's performance as a whole.
- **Speed Measurement:** If you measure the speed of a system across different tasks or components, the harmonic mean will provide a better understanding of the combined performance because it gives greater weight to lower speeds (bottlenecks).

Geometric Mean in System Performance:

- **Benchmarking:** In benchmarks that test system performance across multiple tasks (e.g., SPEC benchmarks), the geometric mean is preferred because it gives a more balanced view of performance when different tasks have widely varying scales of performance (e.g., one task might be 10x faster than another).
- **Growth Rates:** When evaluating system improvements over time, such as performance improvements across different system versions, the geometric mean provides a better indication of average performance scaling, especially when performance grows exponentially.

Example Harmonic Mean

Harmonic Mean

- Is based on the reciprocals of numbers
- Is defined as the reciprocal of the arithmetic mean

• Individual observation $H.M. = \frac{N}{\sum(\frac{1}{x})}$

• Discrete series $H.M. = \frac{N}{\sum(\frac{f}{x})}$

• Continuous series $H.M. = \frac{N}{\sum(\frac{f}{m})}$

x	$\frac{1}{x}$
2574	0.0004
475	0.0021
75	0.0133
5	0.2000
0.8	1.2500
0.08	12.500
0.005	200.000
0.0009	1111.1111
N= 8	1325.0769

$$\begin{aligned}
 H.M. &= \frac{N}{\sum(\frac{1}{x})} \\
 &= \frac{8}{1325.0769} \\
 &= 0.006 \\
 H.M &= 0.006
 \end{aligned}$$

Use Case: Arithmetic Mean in Deep Learning/AI

Loss Function Averaging:

- When training deep learning models, after each batch, the loss is computed. To get the average loss over the entire epoch, the arithmetic mean is used.
- **Example:**
 - `batch_losses` are the individual losses computed for each batch of training data.

```
average_loss = sum(batch_losses) / len(batch_losses)
```

Accuracy Averaging:

- In classification tasks, the arithmetic mean is often used to calculate the **average accuracy** over multiple validation batches or epochs.
- **Example:** If you test a model on 10 different batches and calculate the accuracy for each batch, the arithmetic mean of these values gives the overall accuracy.

Validation/Testing Metrics:

- Metrics such as **mean squared error (MSE)**, **mean absolute error (MAE)**, and **cross-entropy loss** all rely on the arithmetic mean to average the errors over a dataset.

Use Case: Harmonic Mean in Deep Learning/AI

F1 Score for Model Evaluation:

- In classification problems, especially for imbalanced datasets, the **F1 score** (the harmonic mean of precision and recall) is widely used.
- The harmonic mean helps to balance precision and recall, ensuring that both are considered equally, and a poor value in one will heavily influence the final score.
- If precision and recall differ significantly, the harmonic mean provides a more balanced evaluation by giving lower values than the arithmetic mean, penalizing poor performance in either metric.
- **Why Harmonic Mean?:** Precision and recall are rates (ratios of true positives to relevant or retrieved instances), and the harmonic mean is better suited for averaging such rates than the arithmetic mean, which would give an overly optimistic result.

$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Multi-System Throughput:

- In distributed AI systems, where different models or systems work in parallel, throughput (e.g., frames per second, data processed per second) may vary across systems. The harmonic mean is useful here as it penalizes the slower systems and gives a more realistic aggregate of the overall throughput.
- **Example:** If three models are working at 50, 60, and 30 frames per second (FPS), the harmonic mean would penalize the slower one more than the arithmetic mean.

Use Case: Geometric Mean in Deep Learning/AI

Aggregating Benchmark Scores:

- **In DL benchmarks (such as comparing performance across different datasets or tasks)**, the geometric mean is often used to combine scores from different tasks or systems. This ensures that no single extreme value dominates the overall result and that proportional differences are represented.
- **Example:** When evaluating the performance of a model on multiple datasets (e.g., ImageNet, CIFAR-10, COCO), the geometric mean is used to aggregate results, reflecting multiplicative improvements rather than additive.
- **Why Geometric Mean?:** Geometric mean is preferred when dealing with metrics that vary across different magnitudes (e.g., where performance on one dataset may be much higher than on another). This avoids skewing the final metric toward the higher-performing datasets.

Log-Likelihood in Generative Models:

- **In AI applications like Generative Models** (e.g., GANs, VAEs), where **log-likelihoods** are common, the geometric mean is often used to compute the average likelihoods of generated samples.
- **Example:** Suppose you have likelihood values for several generated samples. Instead of averaging them using the arithmetic mean, the geometric mean can better represent the multiplicative relationship between them.